[두산로보틱스] 지능형 로보틱스 엔지니어

비전 기반 사과 수확·품질 분류 자동화 시스템

협동 - 3 : 디지털 트윈 기반 로봇 자동화 시뮬레이션 시스템 구현 프로젝트

C그룹-1조

팀명:

Apple.py

고희태, 이영훈, 노홍동, 조정묵

[멘토] 손미란 강사님



목차

01 프로젝트 개요

02 프로젝트 팀 구성 및 역할

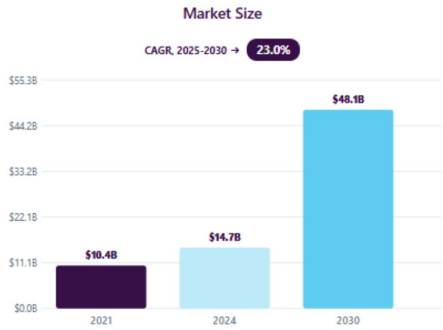
03 프로젝트 수행 절차 및 방법

04 프로젝트 수행 경과

05 자체 평가 의견

1. 프로젝트 주제 및 선정 배경, 기획의도

농업 로봇과 과실 선별 시장이 함께 성장하고 있다



과실 선별 기계 시장

약 43억 달러(2024)
→ 약 84.4억 달러(2035) · CAGR 6.32%

시장 시사점

수확 로봇과 품질 선별을 연결하는
통합 자동화 시스템 필요

약 147억 달러
→ 약 481억 달러

기획 방향: 수확·이송·품질 판정 데이터를 하나의 공정으로 연결

출처: Grand View Research · Market Research Future

1. 프로젝트 주제 및 선정 배경, 기획의도

기존 서비스와 차별화된 프로젝트의 구성

비교 관점	기존 수확 로봇 서비스	우리 프로젝트
중심 목적	현장 수확 자동화·생산성 향상	디지털 트윈 기반 공정·통신 사전 검증
작업 범위	수확 장비 중심	인식 → 수확 → 이송 → 품질검사
데이터 활용	현장 운영 데이터 중심	사과 ID·품질 결과·오류 이력 연결

차별화 방향: 현장 장비 단품이 아닌 공정 전체의 연결과 재현성

1. 프로젝트 주제 및 선정 배경, 기획의도

국내 농업 로봇은 연구를 넘어 현장 실증과 상용화를 준비하고 있다

국내 농업 로봇의 현장 실증·상용화 전환

연구개발 성과를 실제 과수원 운영 시나리오와 연결하는 단계

01 정부 지원 실증

KETI 주도 4년 과제

정부 지원금 74.53억 원

2026년까지 실증 추진

연구 → 현장 검증

02 실증 시나리오

안동 현장 검증

디지털 트윈 농장 연계

다수 이송 로봇 통합 제어

현장·가상 연계

03 상용화 방향

3종 이상 과실 연속 수확

수확·이송 공정의 통합 운영

스마트 과수원 확산 기반

실증 → 확산

시사점: 국내 농업 자동화도 개별 장비를 넘어 현장 운영 시스템 단위로 확대되고 있다.

2. 프로젝트 내용

목표 인식

Base RGB-D 카메라
사과 위치·자세 인식

→

수확·이송

M0617 + 3-finger
Twist & Pull / 2모듈
컨베이어

→

품질 측정

컨베이어 상단 RSD455
OpenCV
RGB·Depth

→

결과 관제

사과 ID·등급 기록
웹서비스·DB 연계

품질 분류 기준

착색률 ≥ 0.80: HIGH(상)
0.60 ≤ 착색률 < 0.80: MEDIUM(중)
착색률 < 0.60: LOW(하)

프로젝트 컨셉

특정 과수 도메인의 센서·로봇·품질·통신 기준을
하나의 운영 구조로 통합해 완전 무인화 기반 구축

Base RGB-D 인식 → 수확 → 컨베이어 이송 → OpenCV 품질 측정 → 등급 결과 표시

3. 활용 장비 및 재료

영역	사용 장비·소프트웨어	주요 용도
실행 환경	Ubuntu 24.04 · Isaac Sim 5.1.0	디지털 트윈·가상 센서
로봇·센서	Doosan M0617 2대 · AGS-001-MTCP 2대 · Base RGB-D 2대 · 컨베이어 상단 RGB-D	수확·깊이·품질 인식
연산·네트워크	GPU PC 2대(RTX 5080) · 개인 PC 2대 · 5포트 Ethernet 스위치(2.5Gbps)	분산 연산·노드 연결
개발 환경	ROS2 Jazzy · Fast DDS · Python · OpenCV · RViz2 · Codex · Claude Code · VS Code	구현·비전·검증
협업·서비스	Slack · Notion · GitHub · Google Drive · HTML 기반 웹서비스·DB	협업·운영 데이터 관리

시뮬레이션·로봇·비전·웹서비스를 하나의 분산 개발 환경으로 연결

4. 프로젝트 구조

V-Model 기반 개발 절차와 검증 대응



설계 항목과 검증 항목을 대응시켜 시스템 완성도를 확인한다.

5. 활용 방안 및 기대 효과

활용 방안

- 현장 도입 전 로봇 경로·충돌·파지 동작 검증
- 수확·이송·품질검사 공정의 반복 실험
- 사과 ID·품질 결과·오류 이력 기반 운영 데이터 관리

기대 효과

- **실제 장비 투입 전 위험요소와 오류 조기 발견**
- **작업자 경험에 의존한 품질 판정 편차 감소**
- **수확부터 품질검사까지 데이터 추적성 확보**
- **완전 무인화를 위한 시스템 기반 마련**

시뮬레이션 기반 사전 검증과 운영 데이터 연결로 완전 무인화 기반을 마련한다.

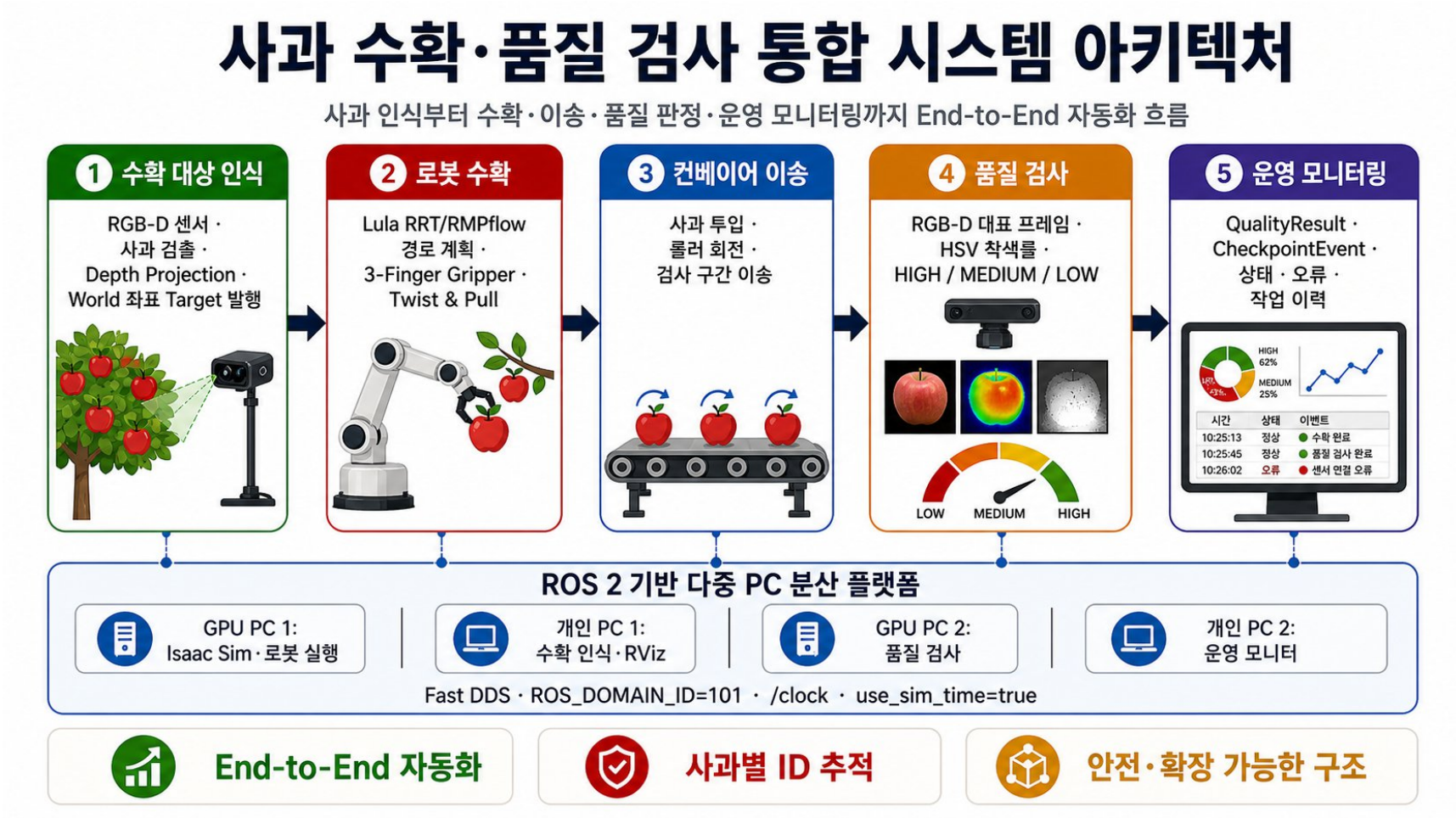
프로젝트 팀 구성 및 역할

참여자	역할	담당 업무		
고희태	팀장	전체 개발 md파일 총괄	로봇 제어 알고리즘	시스템 인터페이스 설계
이영훈	팀원	Twist & Pull 구현	멀티로봇 시스템	
노홍동	팀원	월드 환경 구성	착색률·크기·손상	
조정묵	팀원	품질 분류 구현		
손미란 강사님	멘토	주제 선정 피드백, 프로젝트 질의응답		

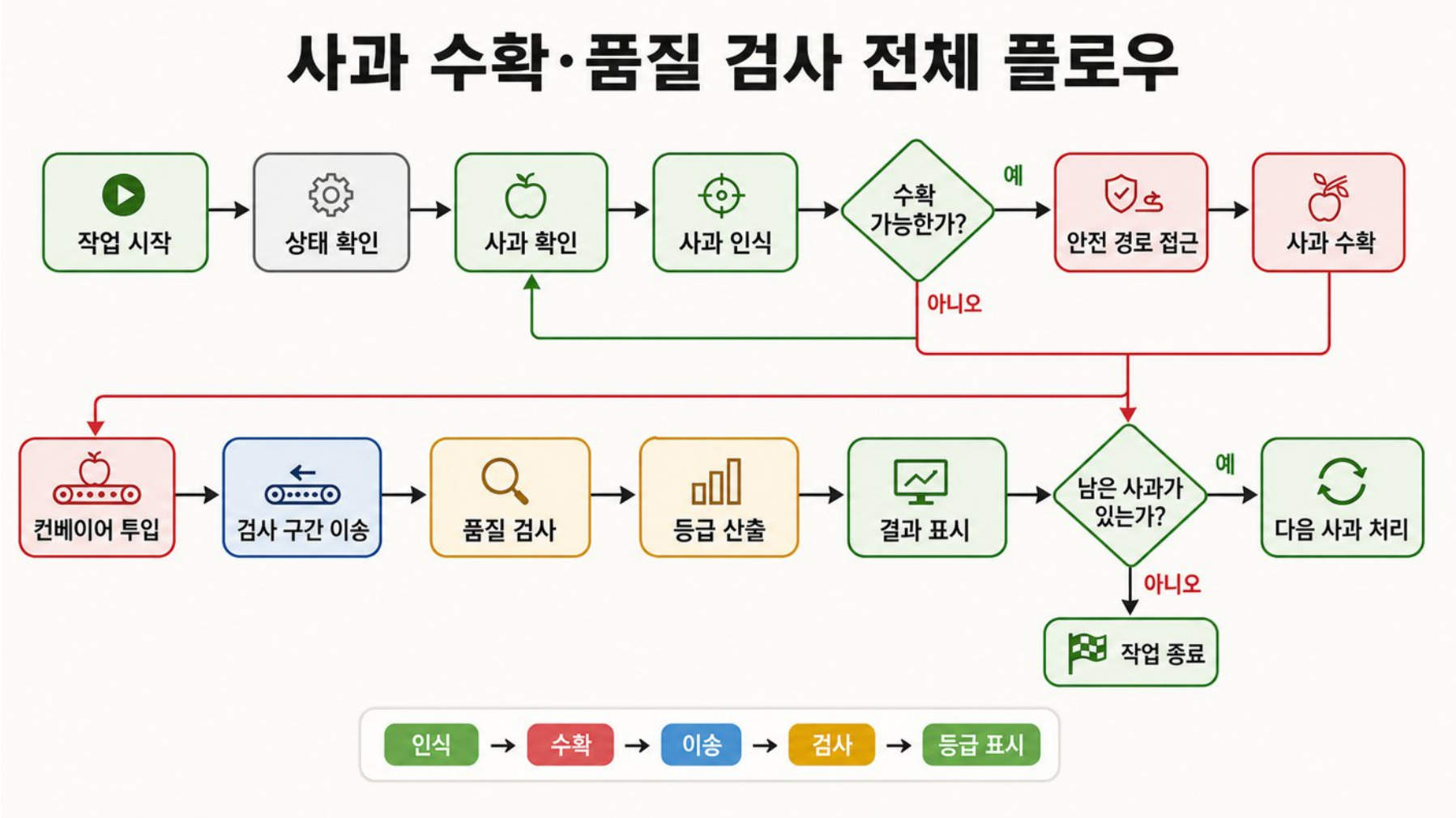
프로젝트 수행 절차 및 방법

구분	기간	활동		비고
사전 기획	08.14(금) ~ 08.17(월)	**조사·요구사항·기획 정리**	**기획 정리**	회의록 근거
환경 검증 및 기술 검증	08.18(화) ~ 8.20(목)	**환경·통신 및 파지 검증**	**센서·에셋 검증**	종료일 확인 필요
단위 개발 및 테스트	8.20(목) ~ 8.25(화)	**수확·이송·품질·관제**		
통합 시스템 구축 및 테스트	08.25(화) ~ 8.27(목)	**TF·모션·결과 수신·UI**		
시연 및 발표	8.27(목)	**통합 시연·자체 평가**	**PDF 제출**	멘토 검토 필요
총 개발기간	8.14(금) ~ 8.27(목) (총 14일)			

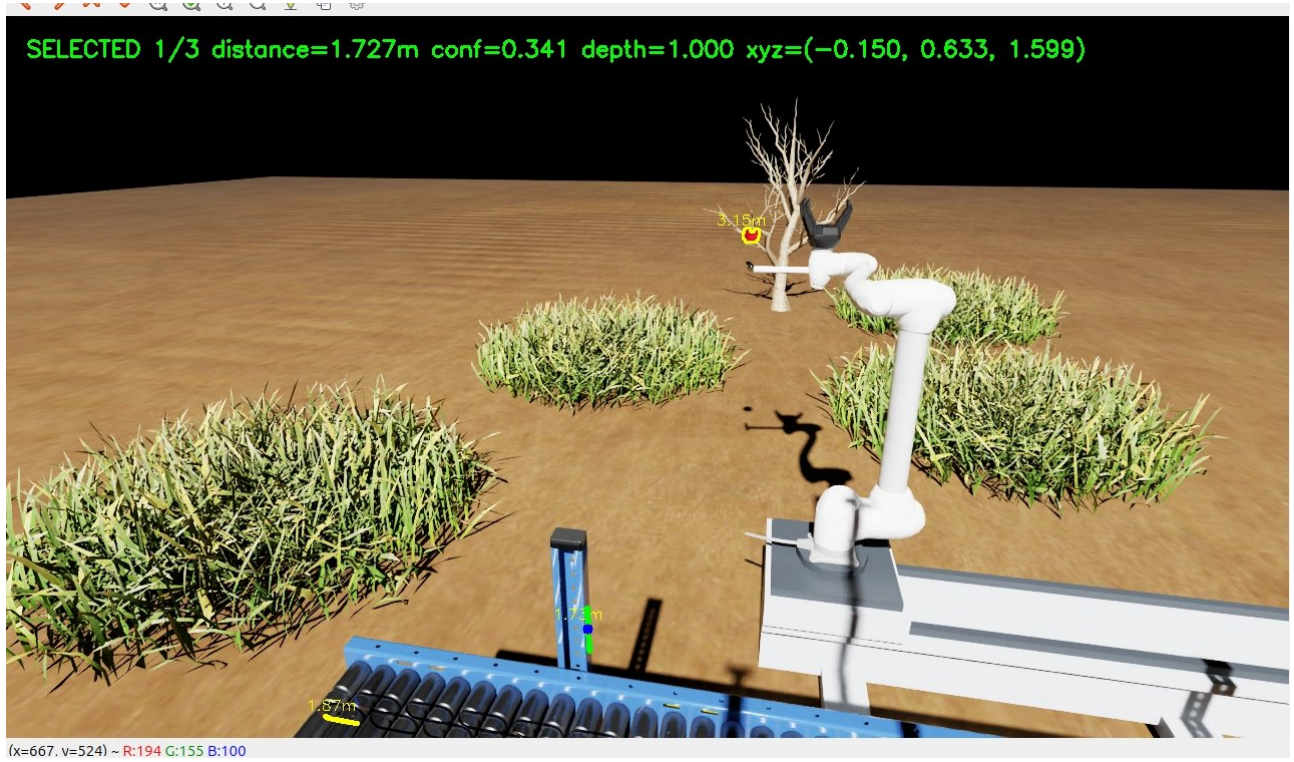
System Architecture | 비즈니스용



Flowchart | 비즈니스용 프로젝트 시나리오



로봇 모델링 | 관절·파지·충돌 모델



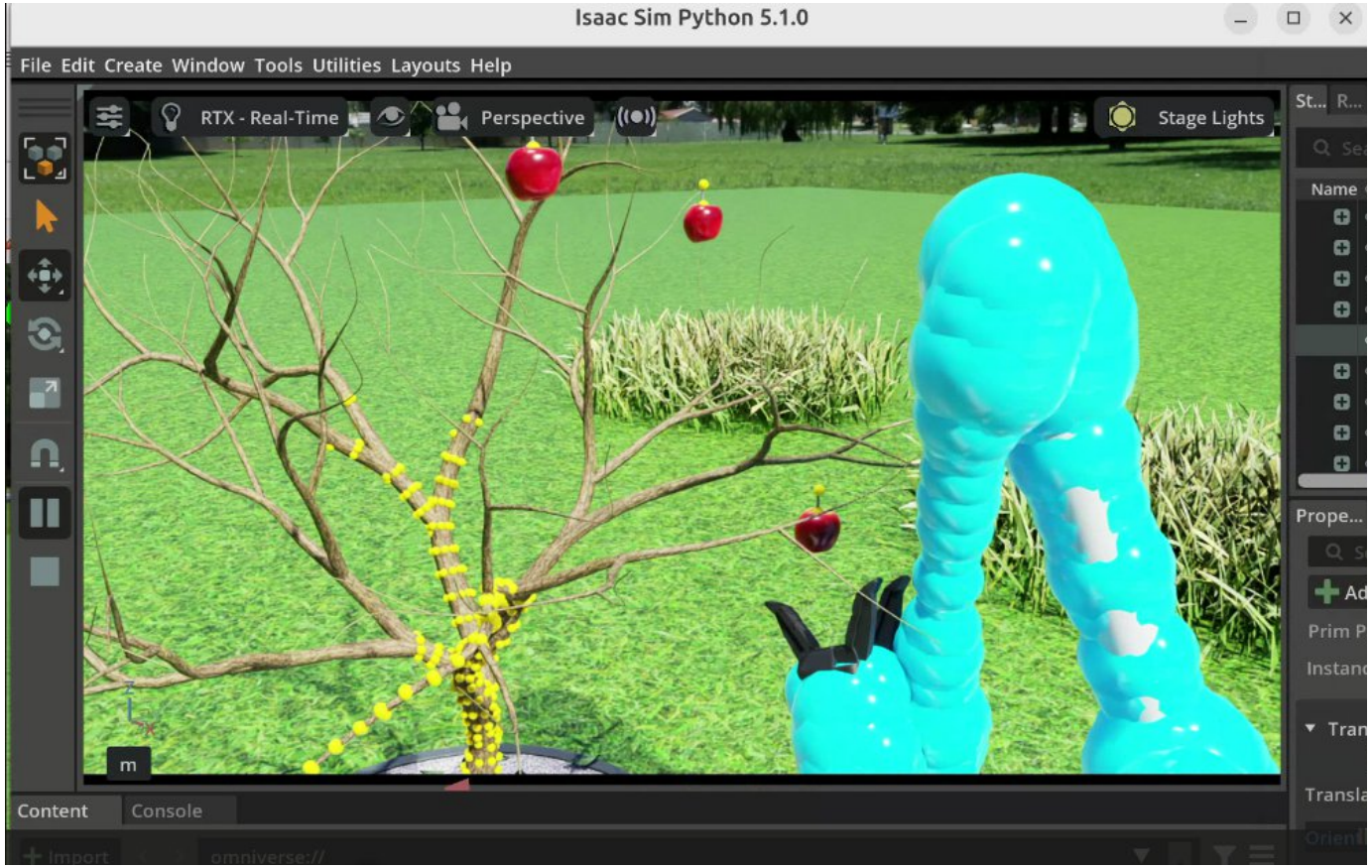
매니퓰레이터 구성

Doosan M0617 6-DOF 2대를 각각 하나의 Articulation으로 구성 – Isaac Sim에서 통합 제어

그리퍼 및 TCP / 물리 설정

3-finger soft gripper 사용,
TCP·관절 drive 제한 조건을 적용해 rigid finger와 compliant contact 기반으로 사과를 파지

로봇 모델링 | 충돌 모델



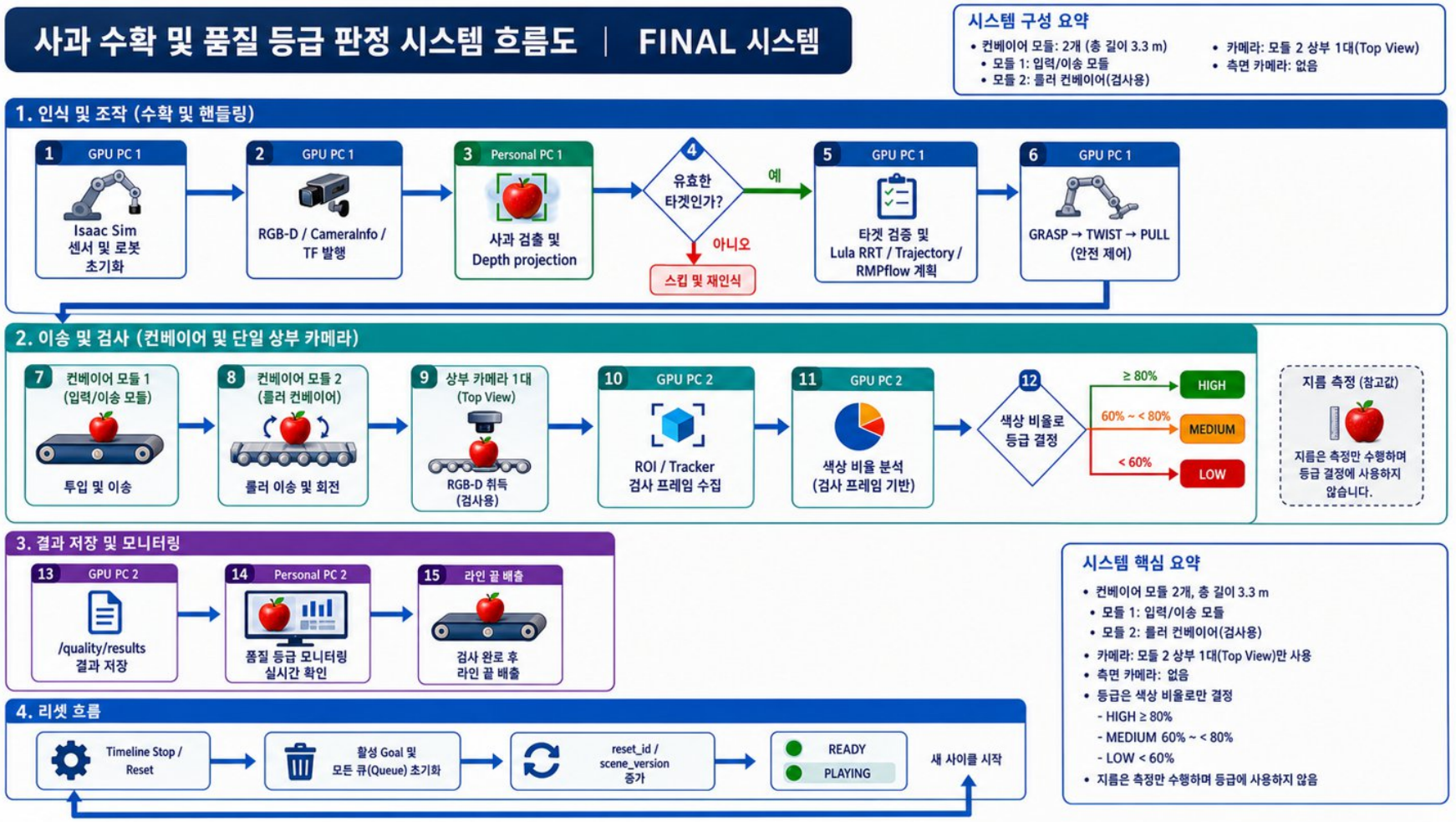
PhysX Collision 적용

M0617 링크와 3-finger 그리퍼의 collision mesh를 사용해 로봇-나무·로봇-사과 접촉을 감지

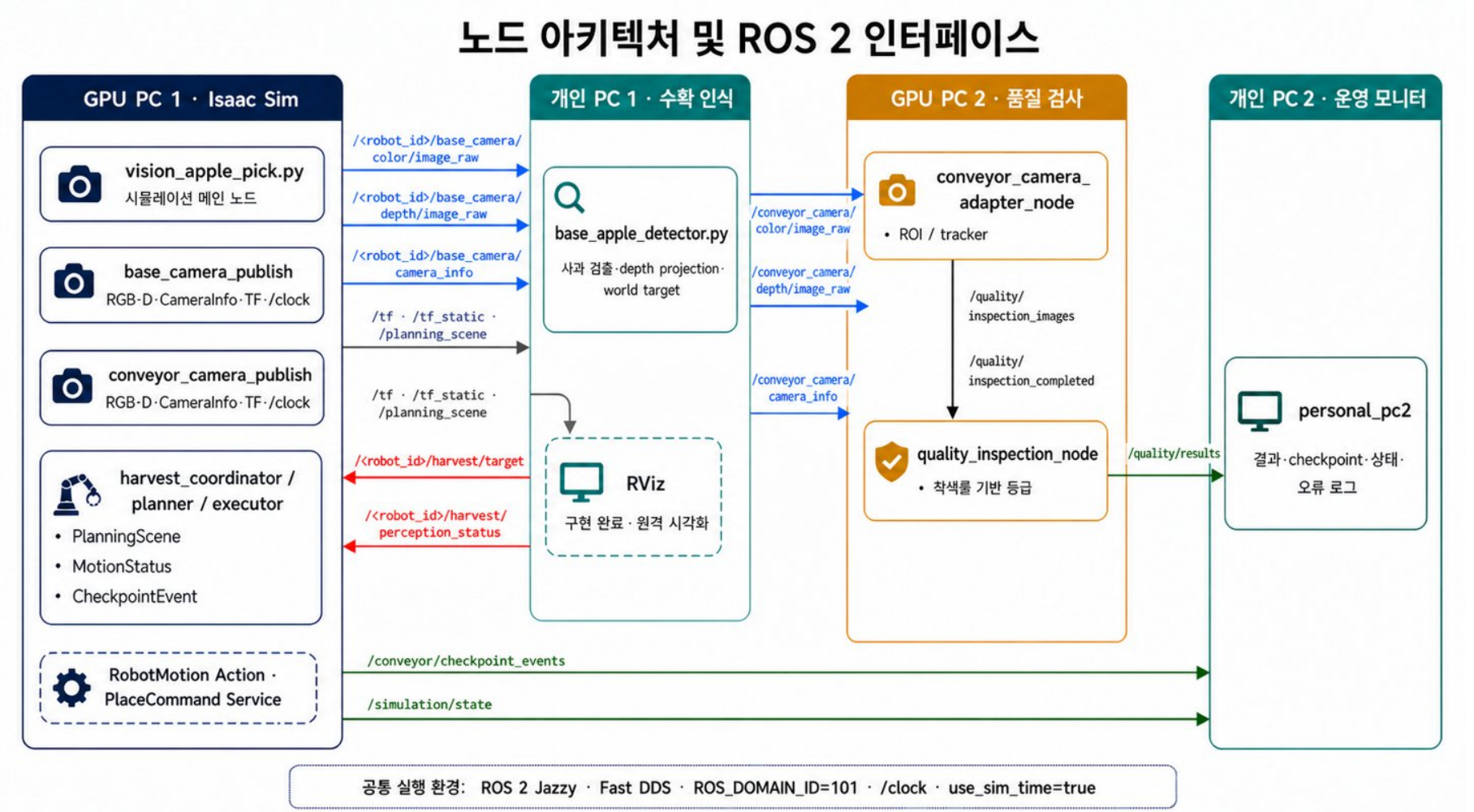
단순 sphere 대신 sphere chain 적용

충돌 과감지 방지, 실제 로봇 형상과 근사
기존 반경 40mm에서 20mm로 축소

Functional Flowchart | 개발자용 수확 처리



Node Architecture & Interface Tree| 개발자용

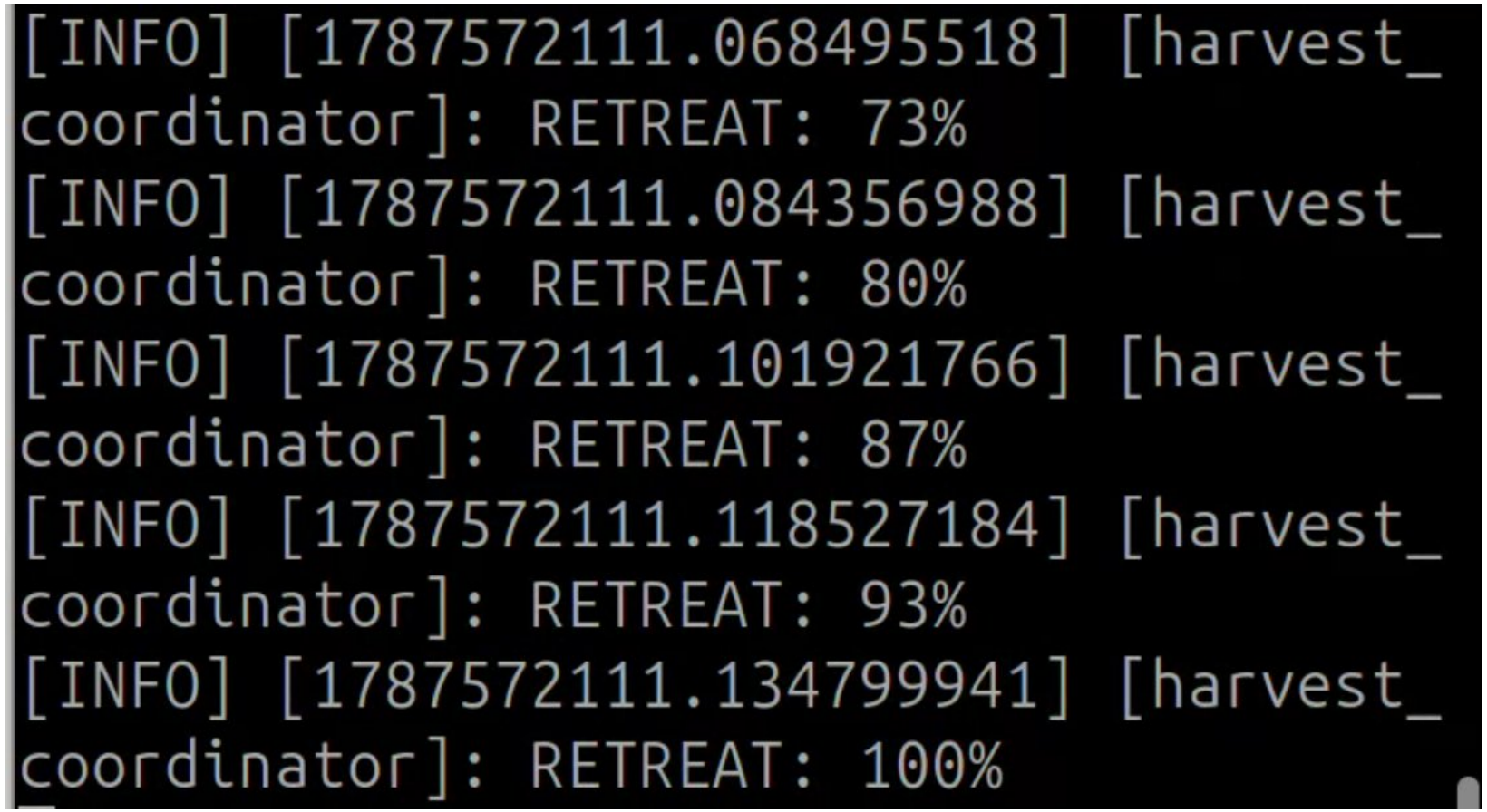


Interface | 데이터·시간·상태 계약

인터페이스	종류 / 형식	핵심 계약
/harvest/target	Topic · HarvestTarget	target_id / reset_id / scene_version
/harvest/robot_motion	Action · RobotMotion	단계 요청 / 진행률 / 성공·오류
/quality/results	Topic · QualityResult	inspection_id / apple_id / 등급·측정
/conveyor/checkpoint_events	Topic · CheckpointEvent	사과 ID / 구간 / sim time

시간 기준: /clock + use_sim_time | ROS_DOMAIN_ID=101 | TF 발행 책임: GPU PC 1

Interface | 데이터·시간·상태 계약



알고리즘 | 충돌 회피와 모션 안정화

IK·경로 계획·trajectory 생성을 분리해 실행 안정성을 높임

Lula RRT

목표 자세의 IK 해를 관절 상태로 변환
장애물을 피하며 waypoint 생성
충돌·관절 제한을 통과한 경로 선택

RMPflow

waypoint를 받아 연속적인 trajectory 생성
속도·가속도와 회피 제약을 반영해 부드럽게 실행
새 목표 전환 시 이전 목표·상태를 정리

실행 전 검사

Lula IK 해 유효성 / joint_1~joint_6 제한 / 전체 링크 충돌 / trajectory 연속성

유효 경로가 없으면 안전 정지

알고리즘 | world 좌표부터 로봇 팔 이동까지

경로 계획 실행 순서

사과의 world 좌표를 로봇 관절 경로와 물리 실행으로 연결

1 좌표 수신

사과 world
좌표 수신

target pose

→

2 Lula IK

joint 각도

q1 ⋯ q6

→

3 Lula RRT

joint_1~joint_6
waypoint 생성

collision-aware path

→

4 RMPflow

trajectory 생성

smooth execution

→

5 PhysX 실행

물리엔진 내
로봇 팔 이동

simulation step

world 좌표 → joint 각도 → joint waypoint → trajectory → 물리 실행

알고리즘 | 롤러 회전으로 사과 전 표면 관측

로봇 파지나 컨베이어 개조 없이, 롤러의 표면 속도와 마찰만으로 사과를 굴려 전 표면을 촬영

평컨베이어 대신 롤러

로봇팔이 잡으면 파지면이 가려짐
컨베이어 구조 변경은 어려움
롤러는 이송과 회전을 동시에 수행

카메라 3대에서 1대로

895프레임에서 3면 동시 검출 0건
동시에 봐도 한 순간은 표면의 36.2%
세 대 모두 위를 보아 시야가 겹침

회전을 시간으로 나눔

이동 1m당 약 13회전, 초당 0.7회전
0.35초 간격으로 8개 시점 채집
시점 사이 약 0.25회전이 벌어짐

현재 구성 · 탑뷰 1대 + 다시점

여러 시점을 모으면 실측 79.5%에 평균 편차 +0.1%p · 표준편차 2.0%p로 수렴 / 회전축 양 극점 때문에 상한 86.5%

단일 시점 착색률은 0.68~0.99로 흔들리지만, 8시점을 합치면 오차가 2.8%p로 줄어듦

알고리즘 | 한 시점의 착색률 산출 파이프라인

한 시점의 RGB와 depth를 짝지어 판정 가능한 표면을 정하고, 그 표면에서 착색 픽셀 비율을 계산

① 사과 영역 검출

HSV 색상 분할 + 윤곽선 추출
Hue 0~25 / 165~179, 채도 20 이상
외접원 채움 0.55로 구조물 배제

② 회전 중 재검출

정반사가 오면 채도 30 아래로 하락
직전 박스 주변에서만 임계 완화
직경 0.55~1.60배 밖이면 기각

③ 유효 표면 확정

최근접 표면에서 6cm 안쪽만 인정
벨트·그림자 혼입은 직경 20~40% 과대
정반사·음영·경계는 분모에서 제외

④ 착색 판정과 ⑤ 시점 통합

Hue 0~10 또는 170~179 · 채도 40 이상 · 밝기 20 이상 / 착색률 = 착색 픽셀 ÷ 판정 가능한 표면 픽셀

ignore_mask를 분모에 남기면 완전히 빨간 사과도 78%로 측정 (깊은 그늘이 표면의 15%)

알고리즘 | 품질 측정과 등급 판단 데이터 분석 및 조절

착색률 = 착색 조건을 만족한 픽셀 수 / 유효한 사과 표면 픽셀 수

Hue·Saturation·Value 센서 값 조절

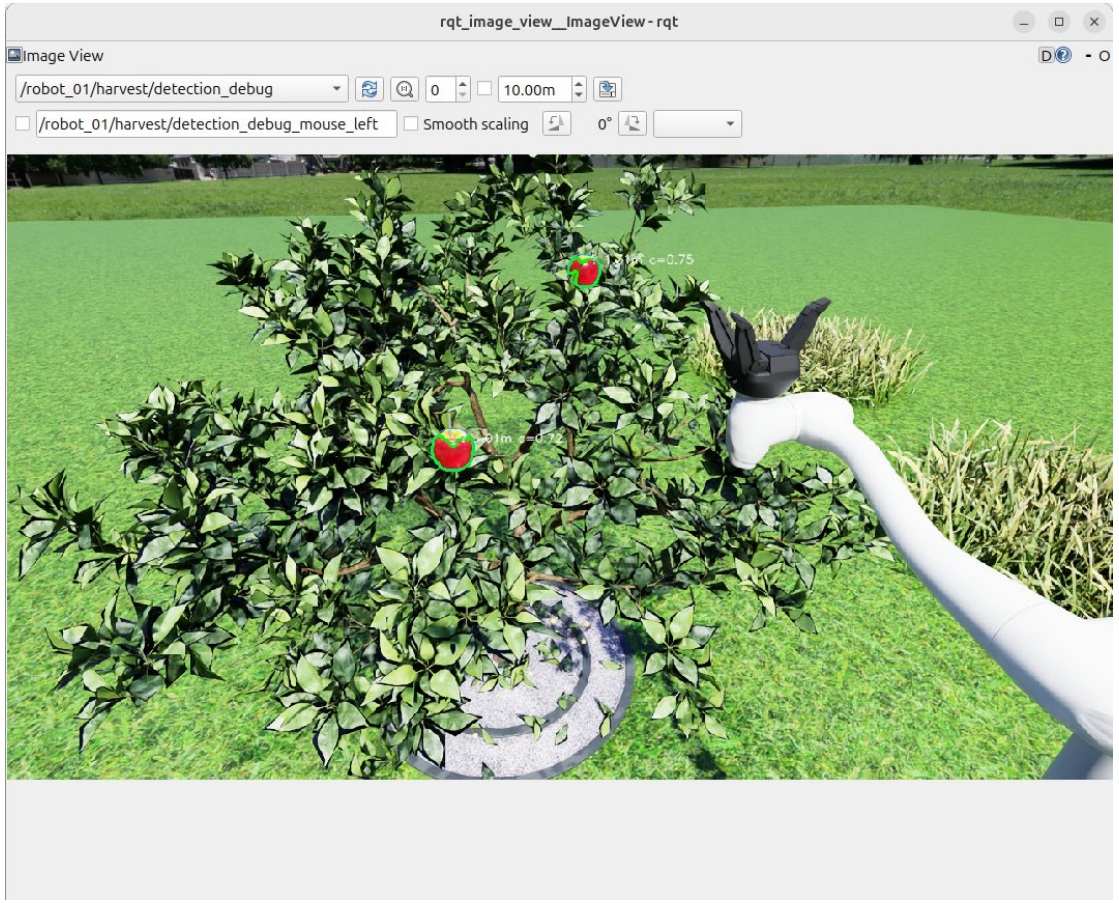
- Hue: `0~10` 또는 `170~179`
- Saturation: `40~255`
- Value: `20~255`

Hue : 적색과 주황을 가르는 지점. 주황 30도, 노랑 60도 (손상면적도 구분하기 위해 두 색을 구분했던 기준)

Saturation : 구형의 사과 색을 구분하기에는 그늘진 면을 측정해야하는데. 채도가 너무 낮으면 그 색상값을 믿을 수 없음.

Value : 너무 어두운 픽셀은 색을 분간할 수 없어서 제외.

알고리즘 | 품질 측정과 등급 판단 데이터 분석 및 조절



HSV 색상 검출 → **depth** 투영 → **world** 좌표 변환 → **target** 발행

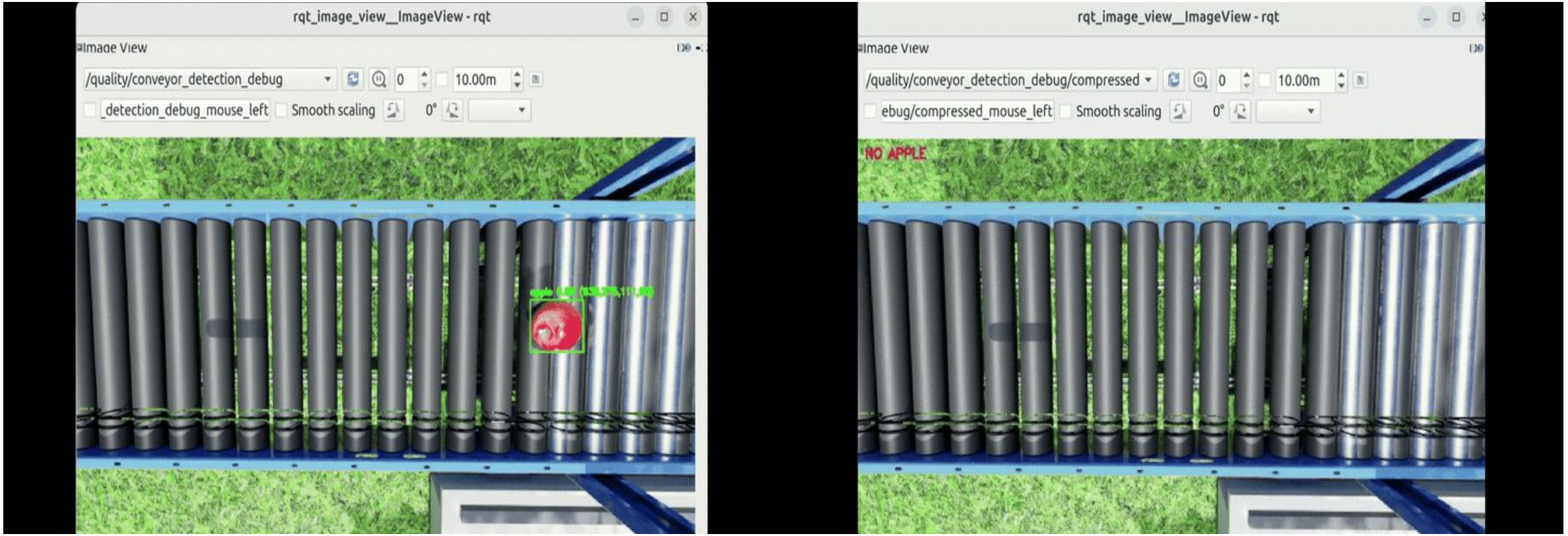
중심점**:** 무게중심 대신 **minEnclosingCircle**

(최소외접원**) —** 잎 가림에 안정적

depth 값**:** 평균 대신 중앙값 **—** 배경 오염 방지

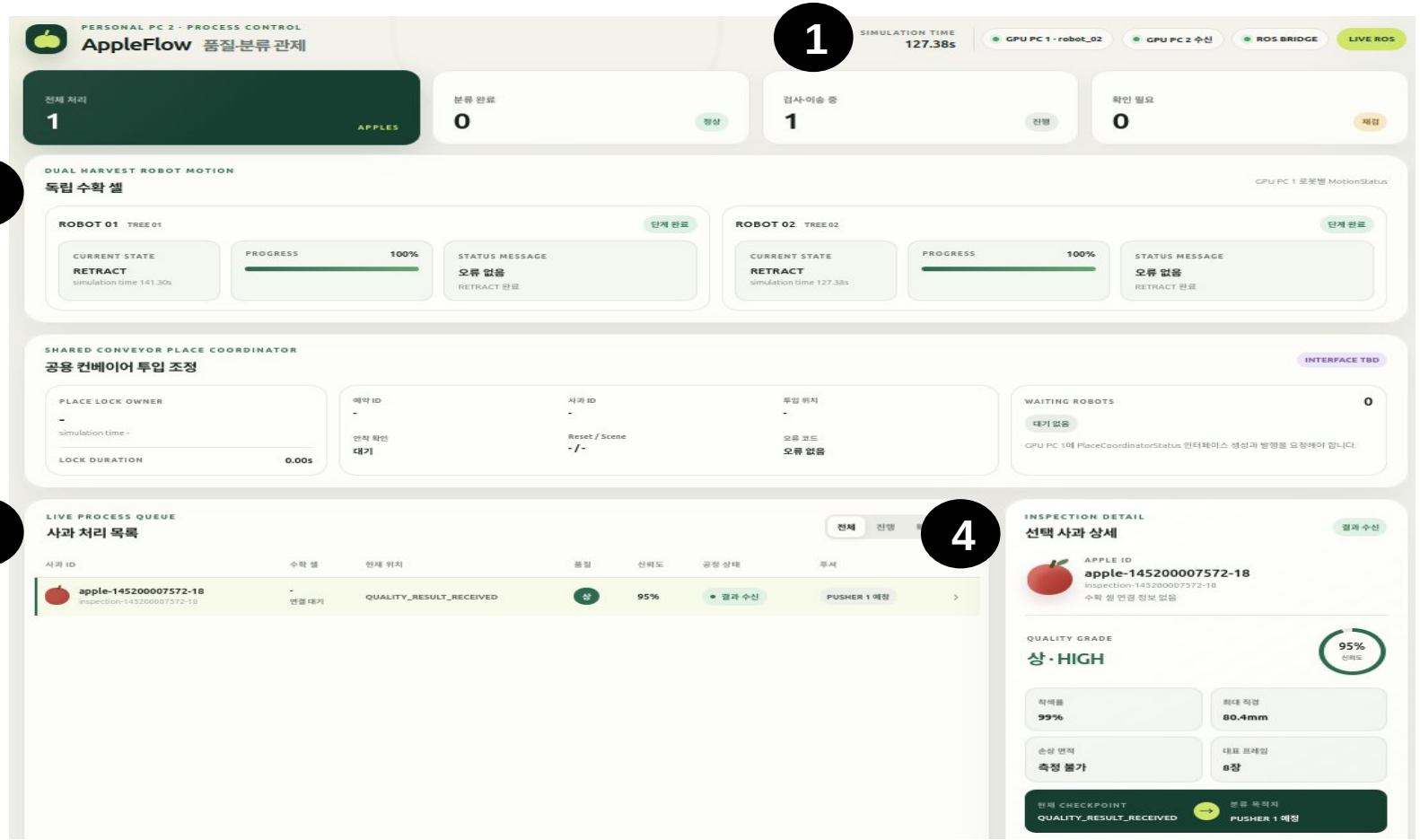
다중 사과 추적**:** 즉시 확정 없이 약 **2**초 안정화 후 **ID** 고정**,** 이후 **100mm** 반경 최근접 매칭만

시스템 부하를 고려한 설계 변경사항



해상도 **1280x760**에서 **640x360**으로 축소**, rgb**는 **JPEG CompressedImage** 형태로 전달
한 프레임당 **2.76MB ->** 수십 **KB** 수준으로 감소

화면 정의서(UI) | 대시보드를 통한 품질·분류 관제



1 연결 상태

PC·ROS 연결 / 시뮬레이션 시간

2 수확 모션

현재 단계 / 진행률 / 오류

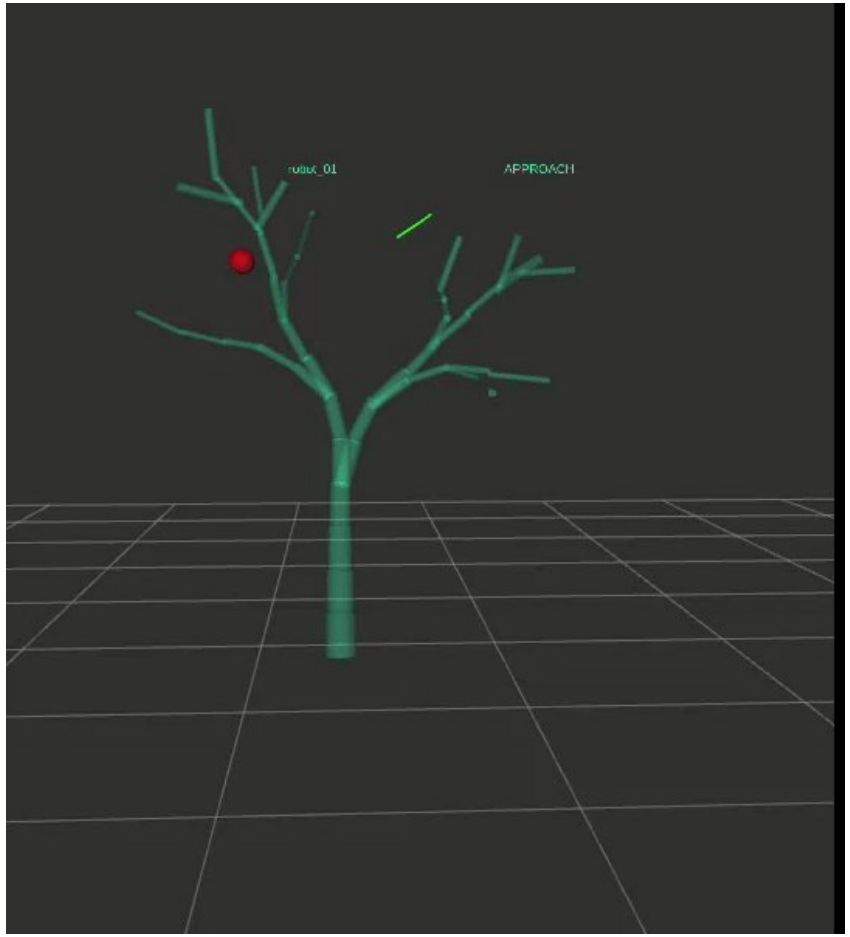
3 사과 처리 목록

사과별 위치 / 품질 / 상태

4 상세 결과

착색률 / 직경 / 프레임

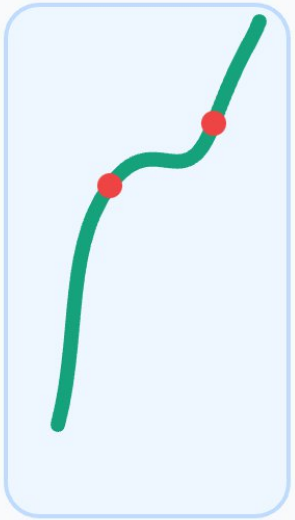
주요 모션 | 매니퓰레이터 이동



RViz2 motion planning 시각화

목표점부터 로봇 경로 실행까지 화면에서 확인

1. **world 좌표 기준 목표점 설정**
   사과 위치·자세를 목표 pose로 전달
2. **IK 결과로 관절 목표 자세 계산**
   Lula IK가 joint_1~joint_6 각도를 계산
3. **waypoint와 충돌 상태 확인**
   Lula RRT 경로를 RViz2에서 시각적으로 검토
4. **연속 trajectory 실행 전달**
   RMPflow trajectory를 PhysX에서 실행



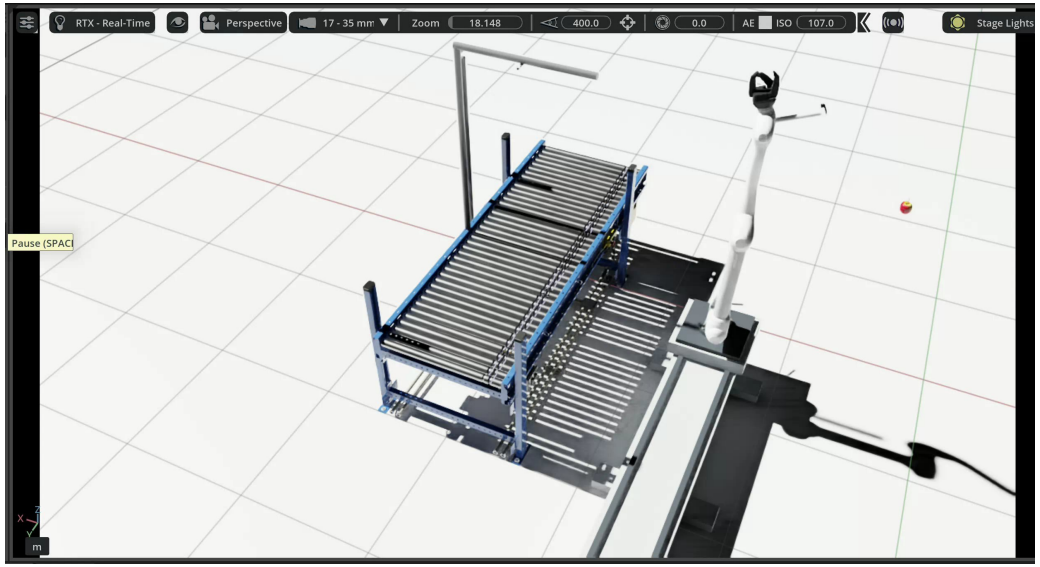
path / pose / collision

주요 모션 | 수확 시퀀스와 이송



Twist & Pull

손목 45도/1초 회전. 당김 속도 50mm/s, 최대 거리 100mm

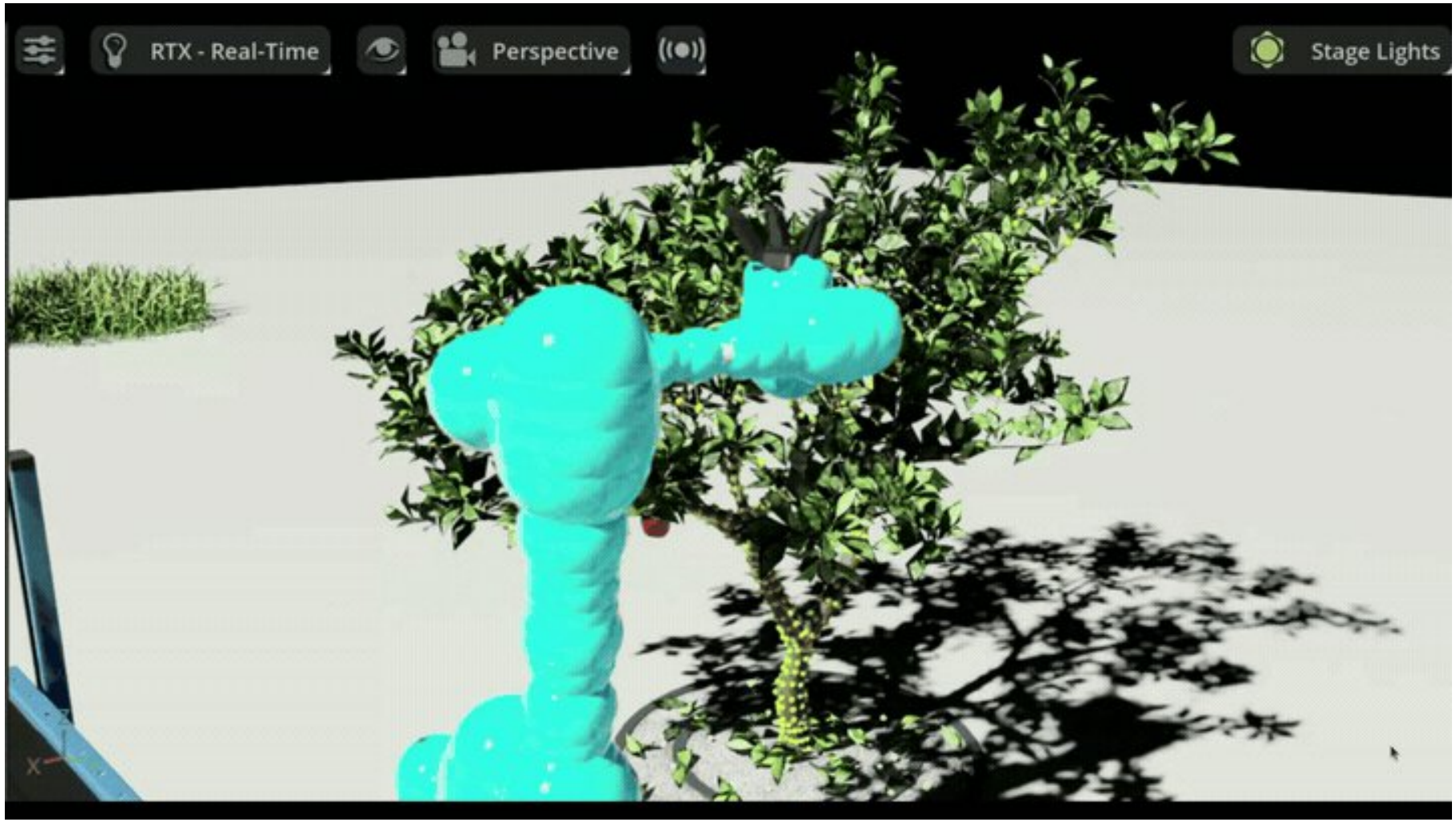


운반 → 배치 → 개방 → 후퇴

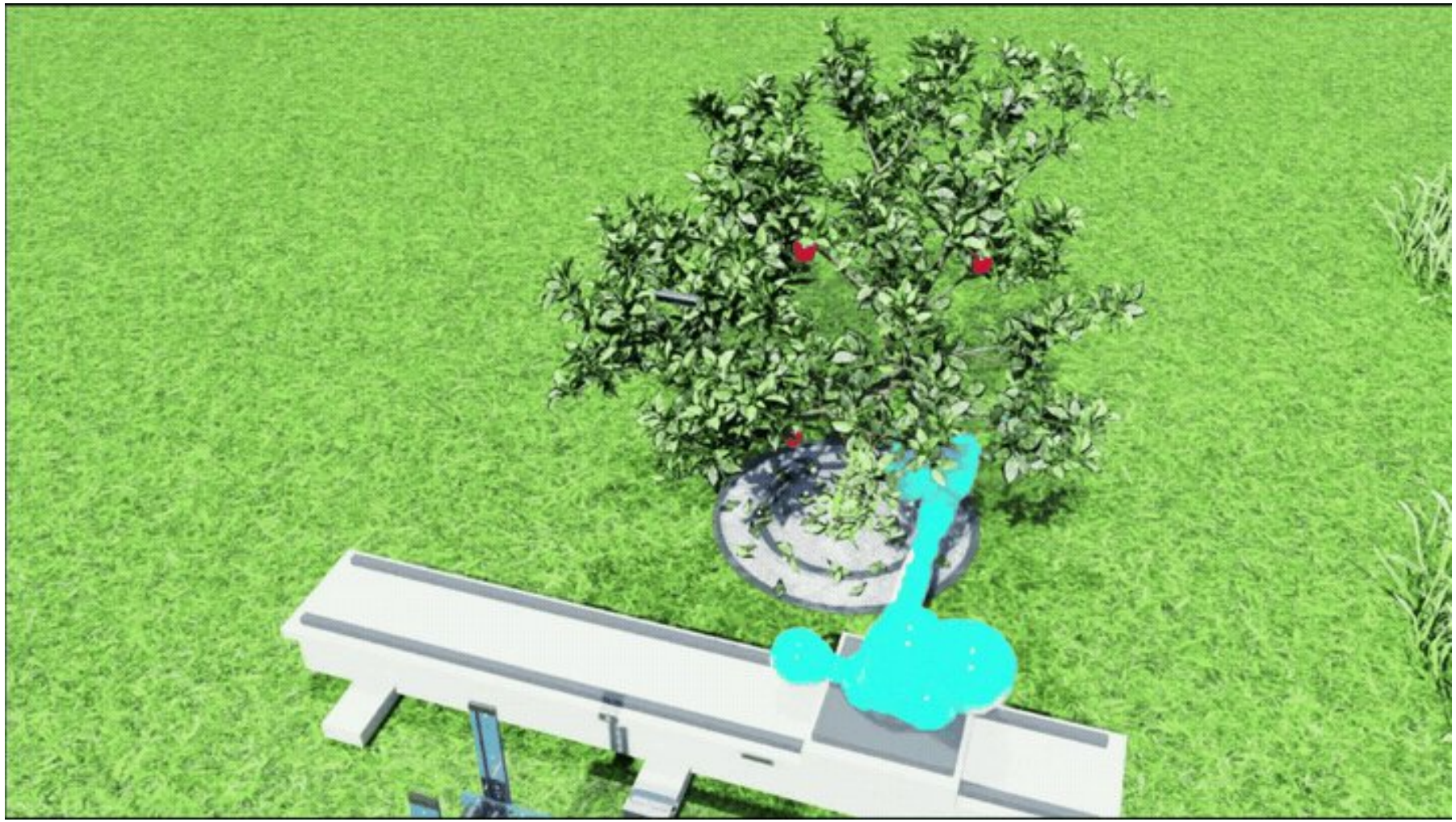
벨트 상면 30mm 이내에서 그리퍼 개방

사과의 컨베이어 안착·이송 여부와 로봇의 안전 후퇴 확인

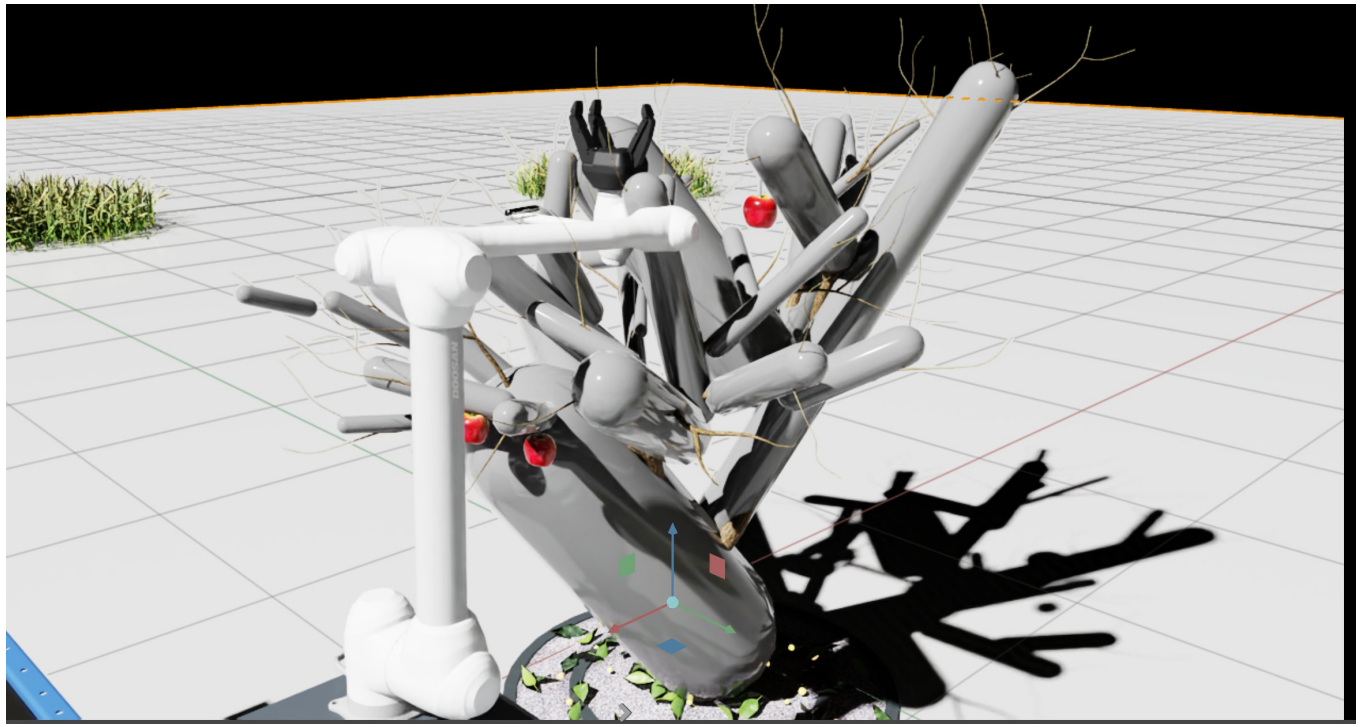
로봇 모션 개선



로봇 모션 개선



비정형 자연환경 대응



복잡한 형상의 자연 환경에 대응

비정형 환경에서의 motion planning을 통해
다양한 collision 형상에 대응할 기반 마련

Twist & Pull



단순 pull이 아닌, Twist & Pull

Breakable Joint를 도입하고, Force와 Torque로 분리 조건을 설정
-> 다양한 분리 조건에 대응 가능

멀티로봇 시스템



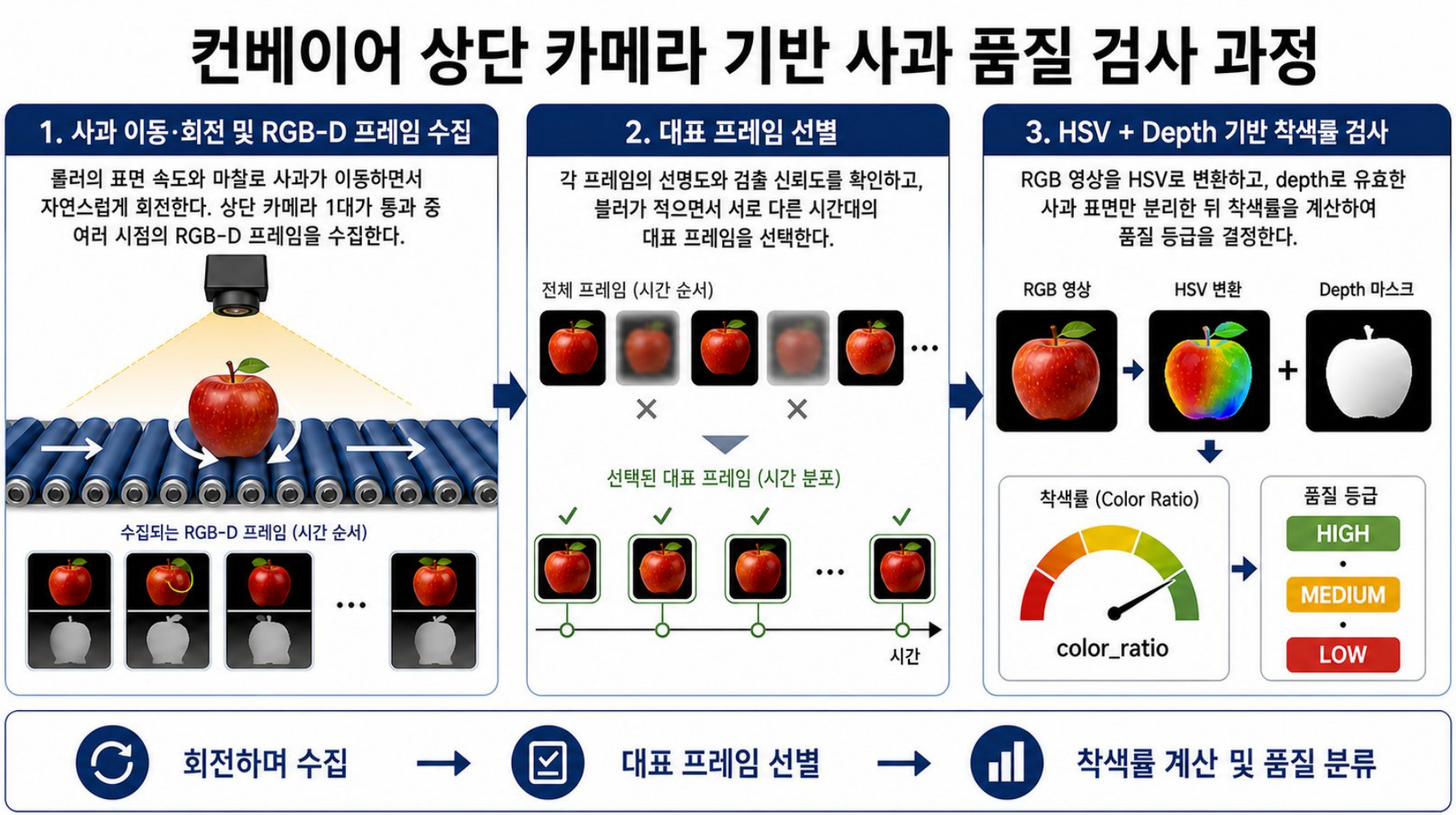
통합 환경에 대비한 멀티로봇 시스템

작업 시간 / 공간적인 효율을 고려하여 한 컨베이어를 공유하는 두 개의 로봇 구현

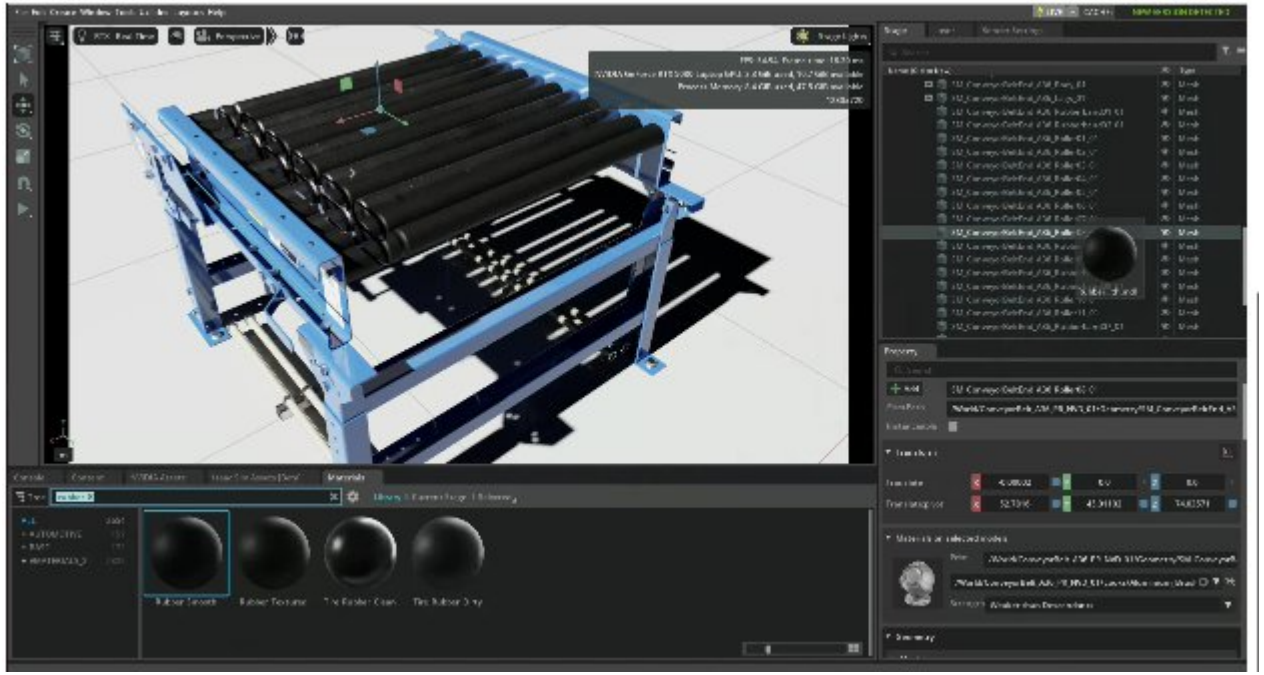
Namespace 분리

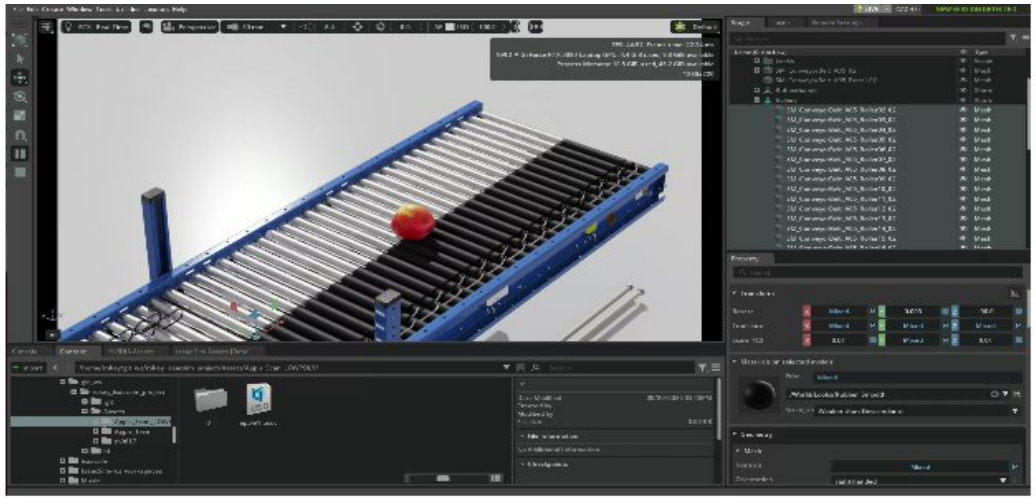
싱글로봇에서 멀티로봇으로 확장하면서 로봇별 namespace 사용

품질 검사 및 분류


컨베이어 상단 카메라 기반 사과 품질 검사 과정
1. 사과 이동·회전 및 RGB-D 프레임 수집
롤러의 표면 속도와 마찰로 사과가 이동하면서 자연스럽게 회전한다. 상단 카메라 1대가 통과 중 여러 시점의 RGB-D 프레임을 수집한다.
수집되는 RGB-D 프레임 (시간 순서)
2. 대표 프레임 선별
각 프레임의 선명도와 검출 신뢰도를 확인하고, 블러가 적으면서 서로 다른 시간대의 대표 프레임을 선택한다.
전체 프레임 (시간 순서)
선택된 대표 프레임 (시간 분포)
시간
3. HSV + Depth 기반 착색률 검사
RGB 영상을 HSV로 변환하고, depth로 유효한 사과 표면만 분리한 뒤 착색률을 계산하여 품질 등급을 결정한다.
RGB 영상
HSV 변환
Depth 마스크
착색률 (Color Ratio)
color_ratio
품질 등급
HIGH
MEDIUM
LOW
회전하며 수집
대표 프레임 선별
착색률 계산 및 품질 분류

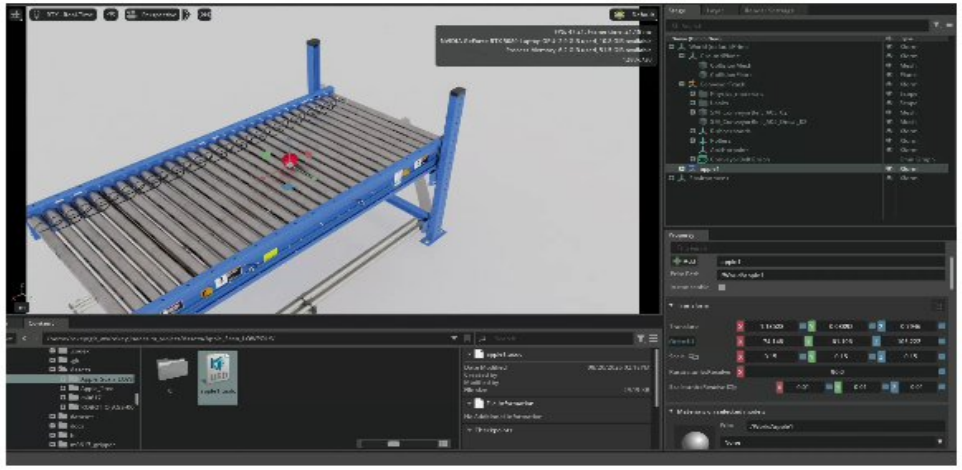
평컨베이어랑 롤러 중 롤러 선택
→ 사과가 손상되지 않을까?
해결 방안
고무를 감싼 형태의 롤러 혹은 롤러 자체가 고무 재질인 롤러로 채택하기로 함.





문제 파악 - 사과가 롤러 위에서 잘 돌아가지 않는다.

스케일 1인 컨베이어 벨트 위에서 현실 크기 사과가 잘 굴러가는지 확인

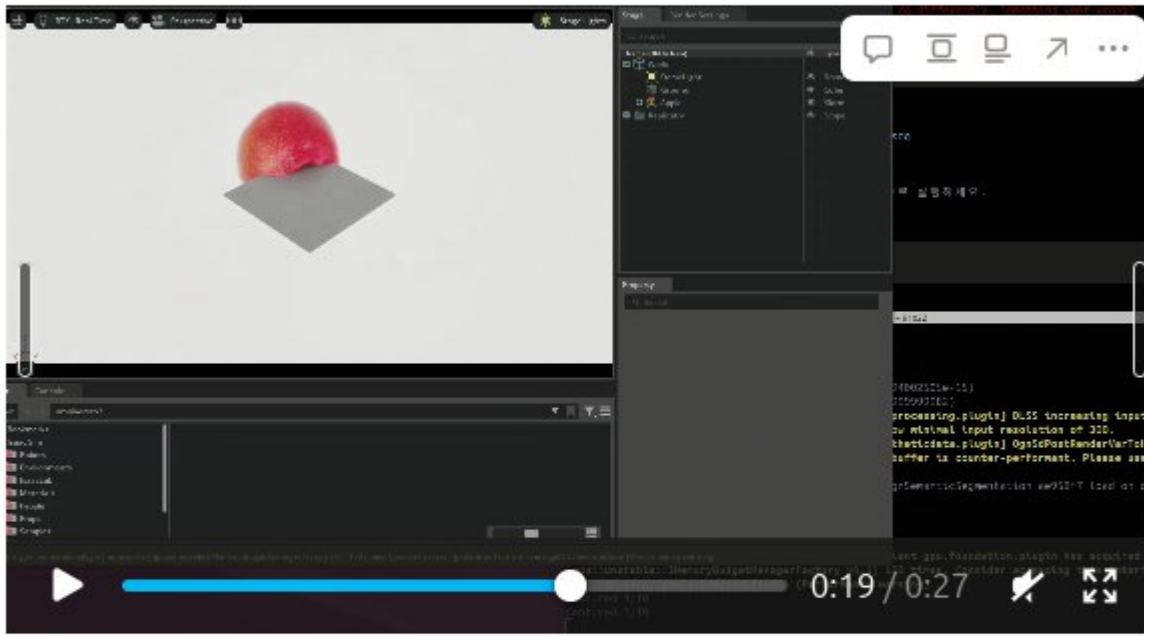


컨베이어 속도를 통해 사과가 잘 굴러가는 지점을 모색 **-> 0.4**

0.4 미만은 사과가 같은 자리에서 헛도는 시간이 너무 길어 다음 투입되는 사과와 함께 지나가게 되는 경우 발생

또 속도를 **0.4**보다 너무 높게하면 사과 색을 구분하는 카메라를 너무 빨리 지나가거나 사과가 굴러가지 못해서 여러면을 찍지 못하는 경우가 생김

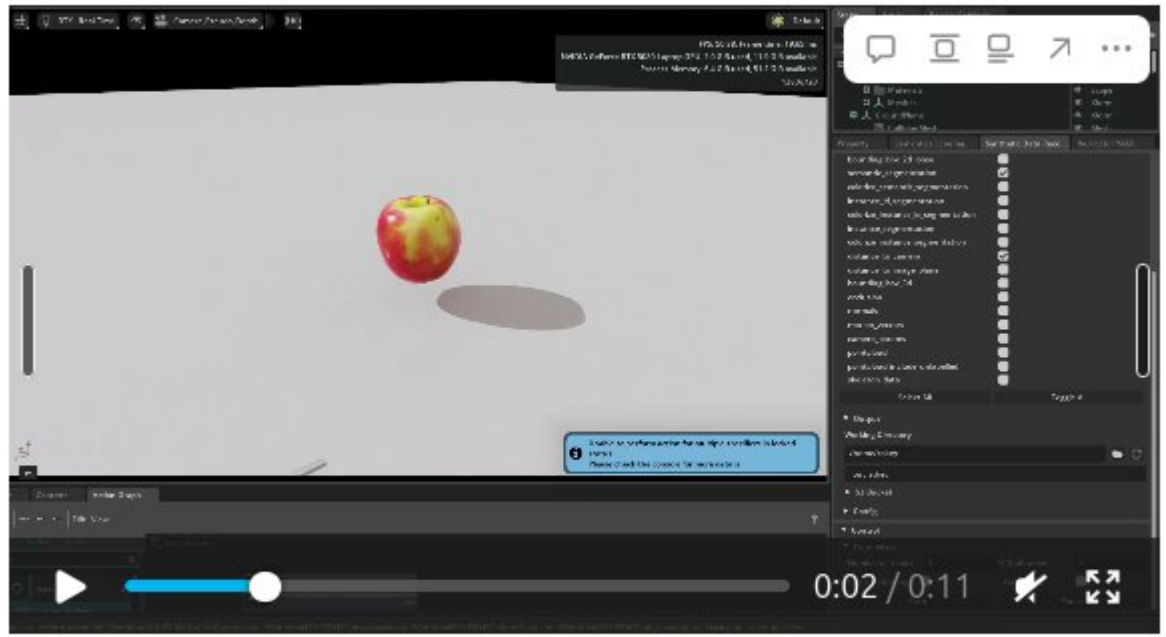
품질분류 진행 기록



gpt를 이용해서 만든 데이터 수집 프로그램 작동 영상

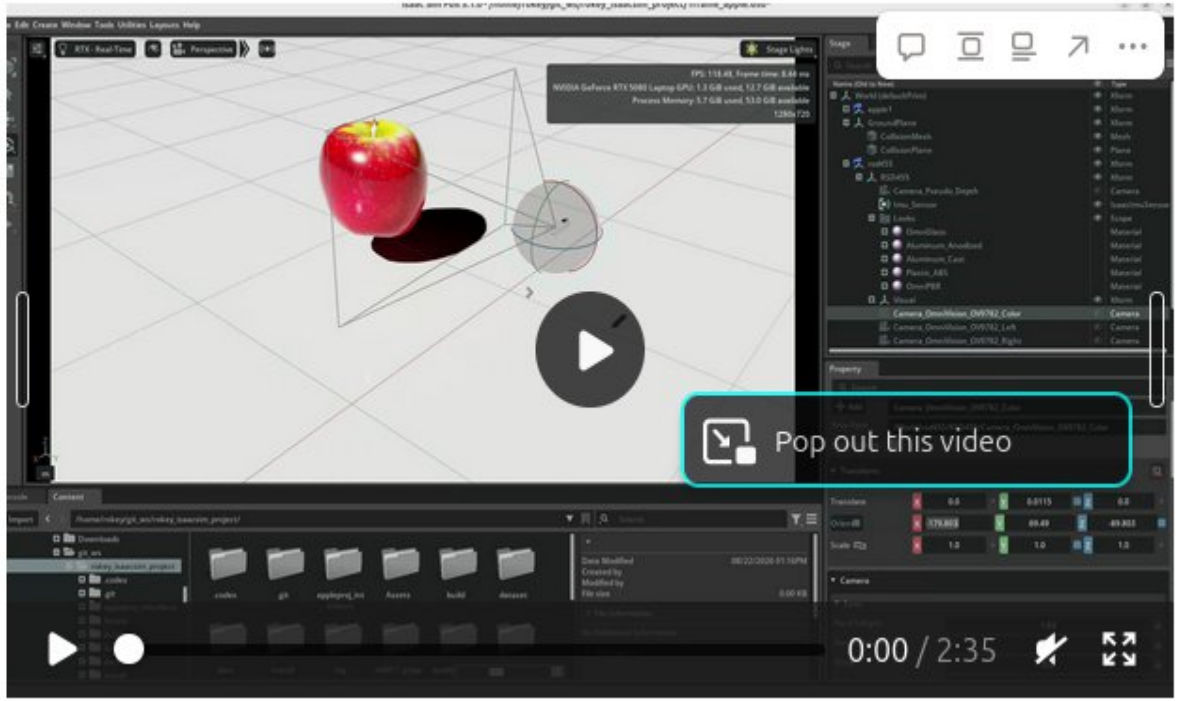
- 위치 변환이나 여러가지 변수를 직접 정하지 않아서 gpt가 임의 설정함
- 카메라 위치가 실제 사과를 찍지 못하게 좌표가 설정 되어 있음

직접 설계

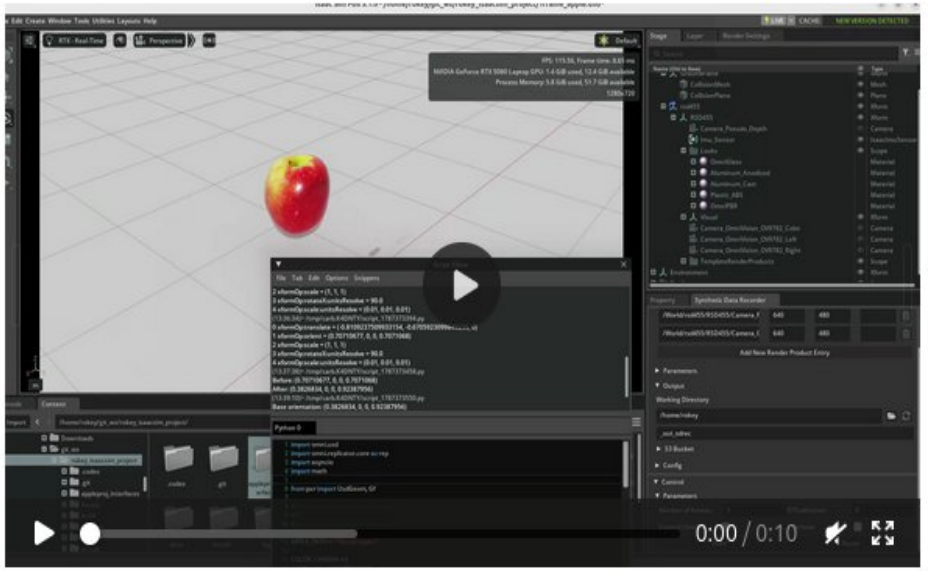


카메라와 사과만 두고 1 frame 촬영

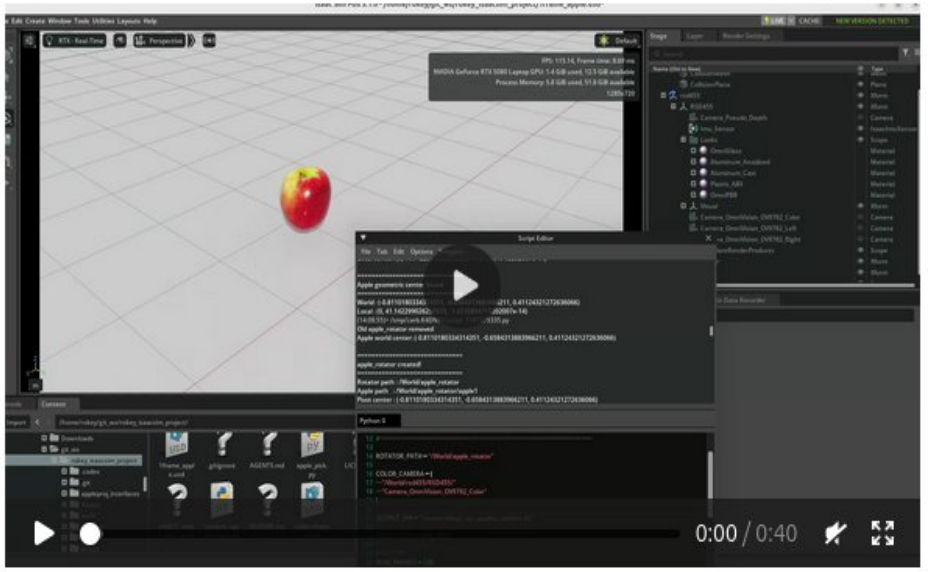
카메라 각도 조정



script editor 관리

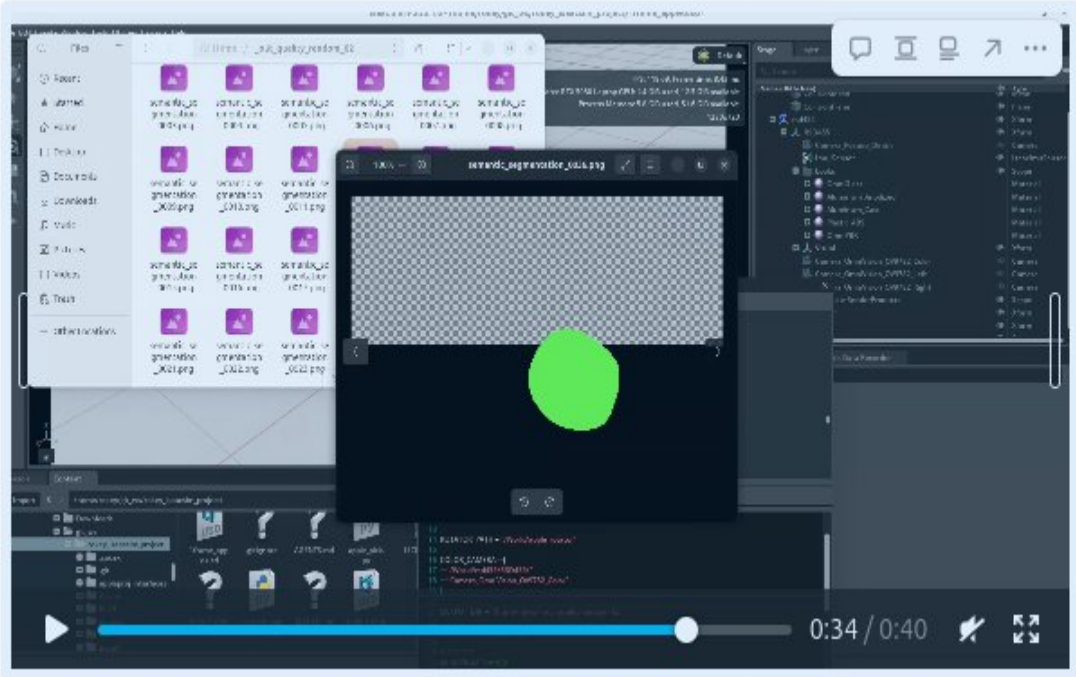


3축 랜덤 사과 100장 촬영 영상



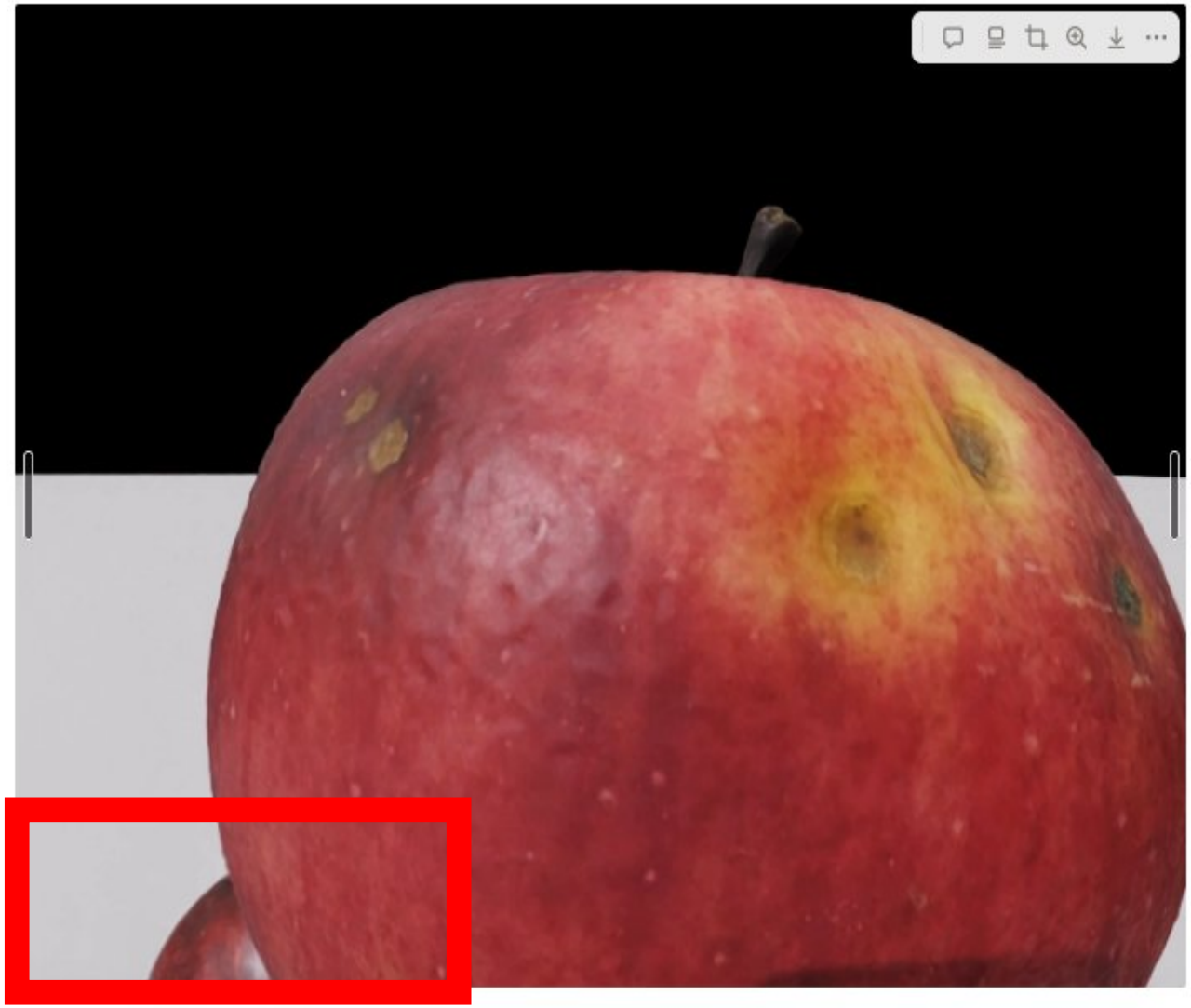
사과의 에셋에 저장된 사과의 축이 눈에 보이는 사과 중심이 아닌 문제가 있어**. rotate** 좌표를 다시 설정한 후 **3**축으로 랜덤하게 돌아가도록 설정

3축 랜덤 사과 100장 촬영 영상

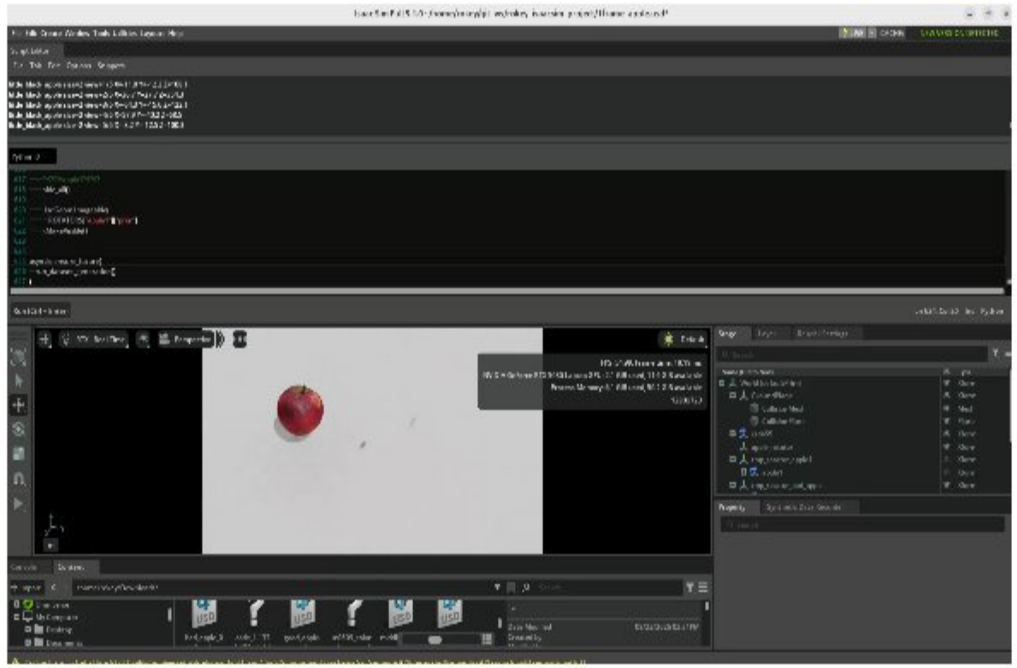


여러 사과를 찍을때 생긴 문제

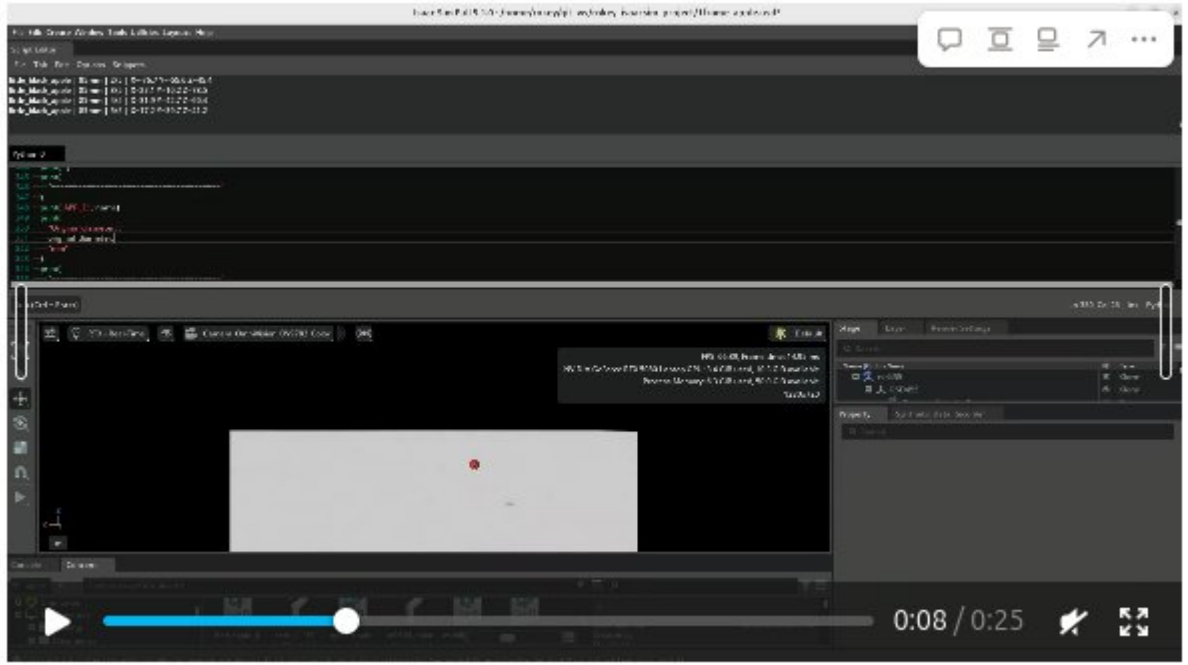
- 이전에 찍었던 사과가 사라지지 않아서 새로 찍을 사과가 가려짐
- 사과의 위치가 중심점으로 이동하지 않아서 사과의 전체 모습을 볼 수 없음



수정 후 진행 영상

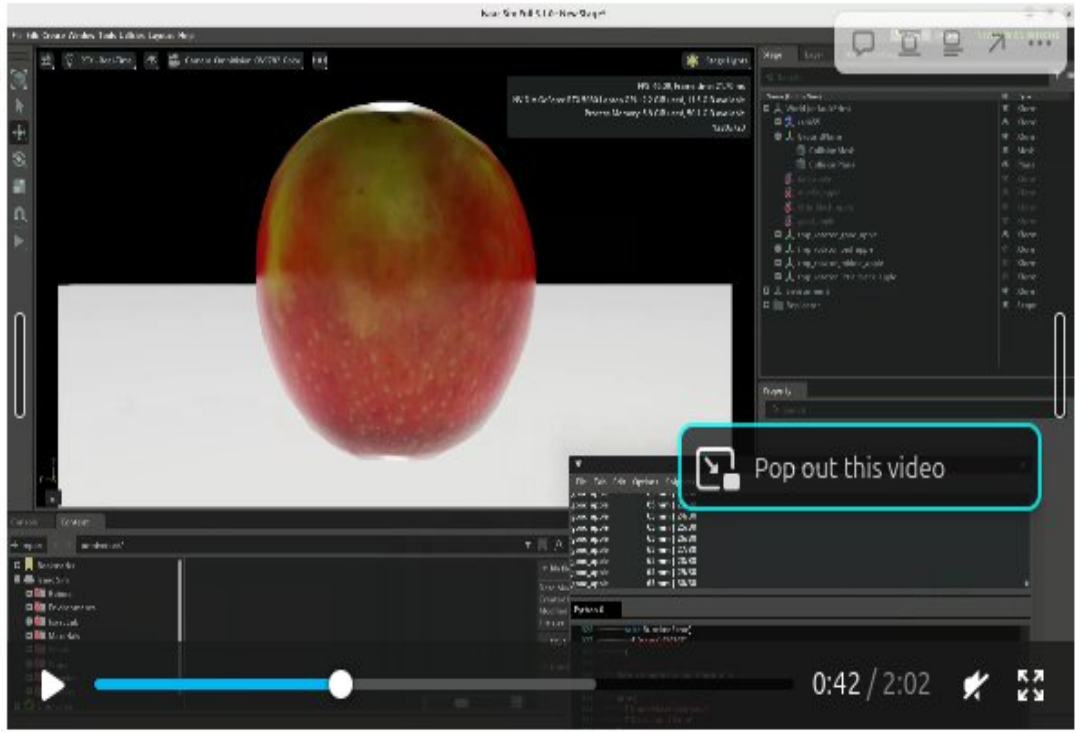


문제 - 카메라가 멀어서 사과가 너무 작게 보임

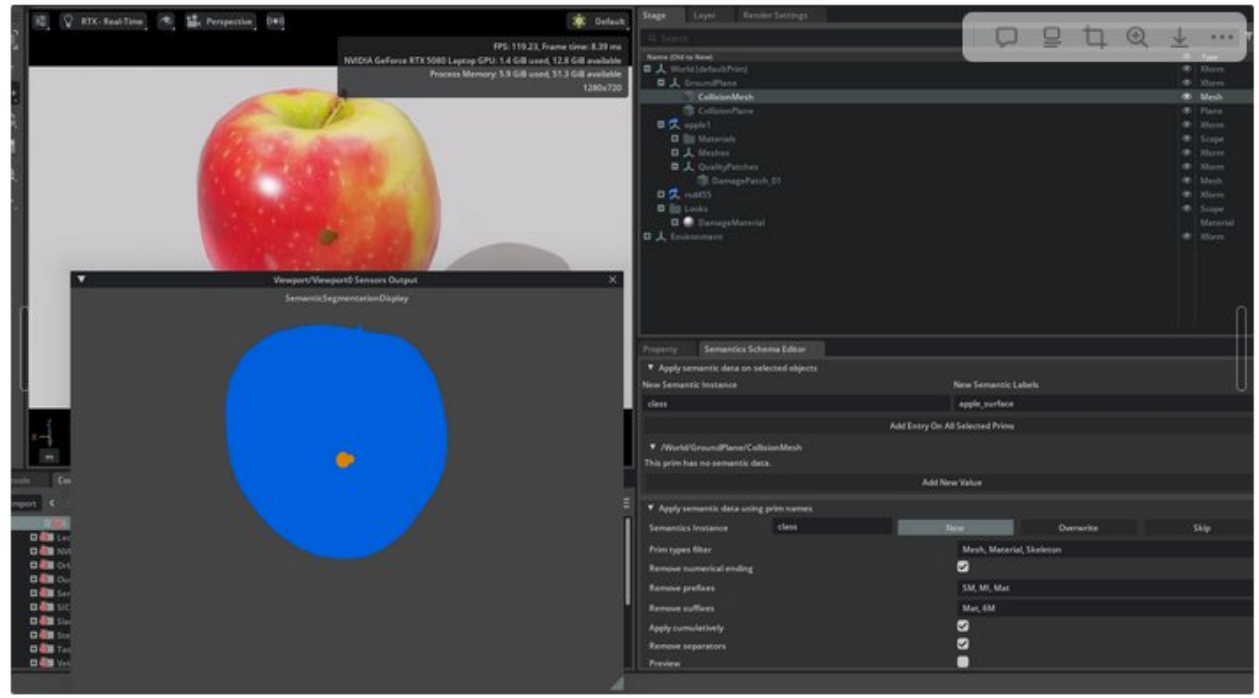


사과 크기를 실제 크기로 맞추고 크기 분류 기준을 학습할 수 있도록 크기를 변경하면서 촬영

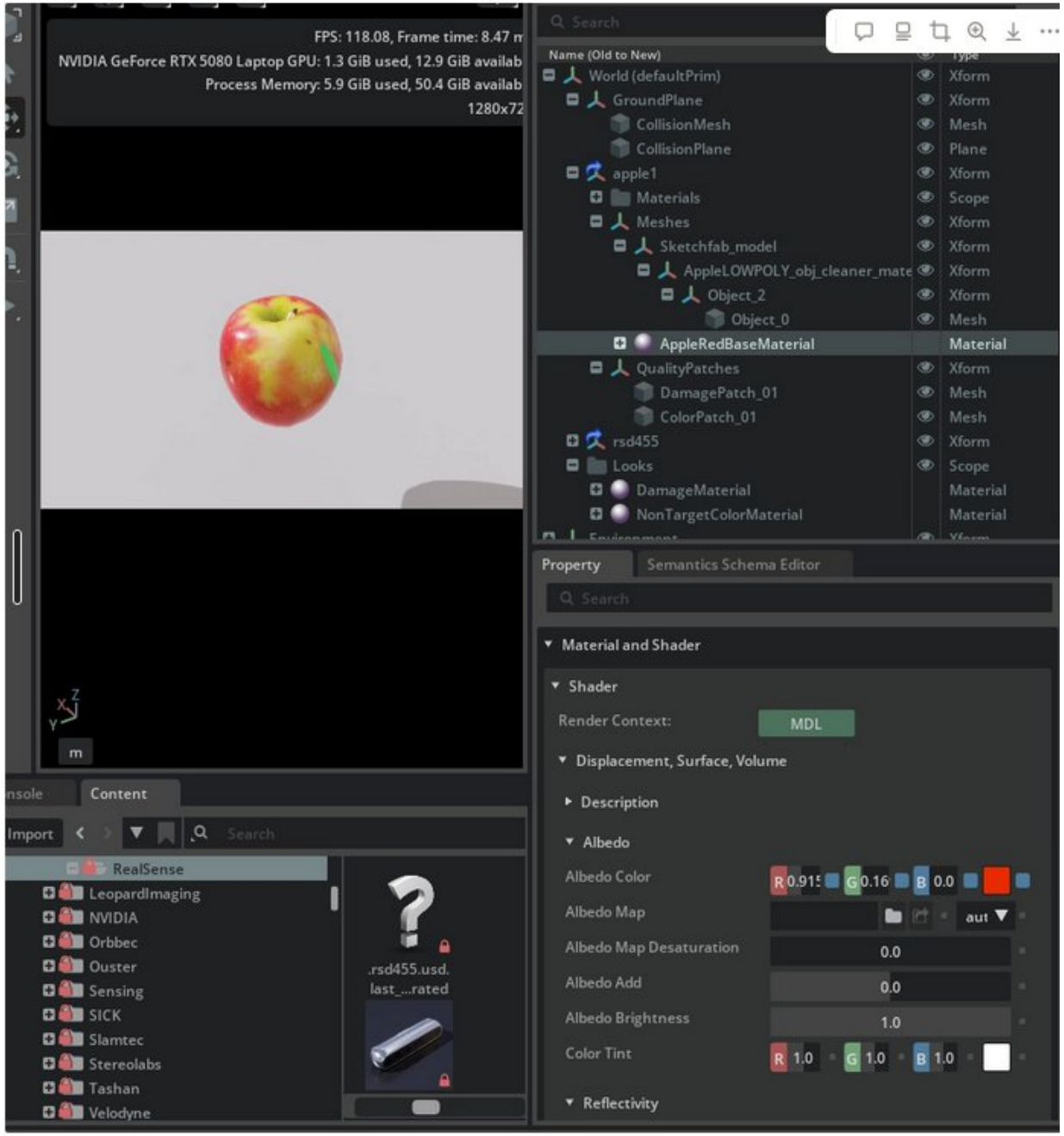
최종 사진찍는 과정



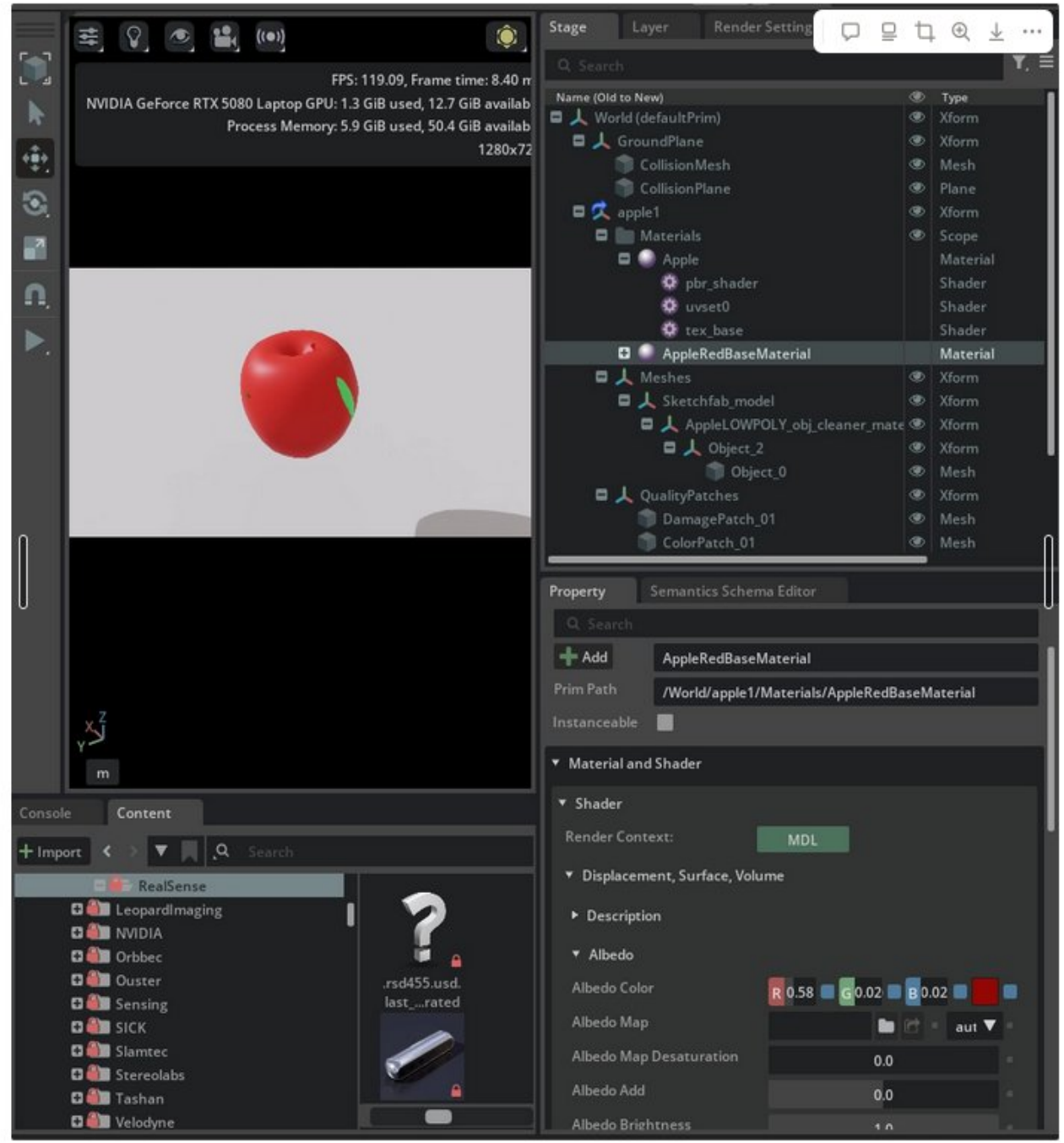
damage masking



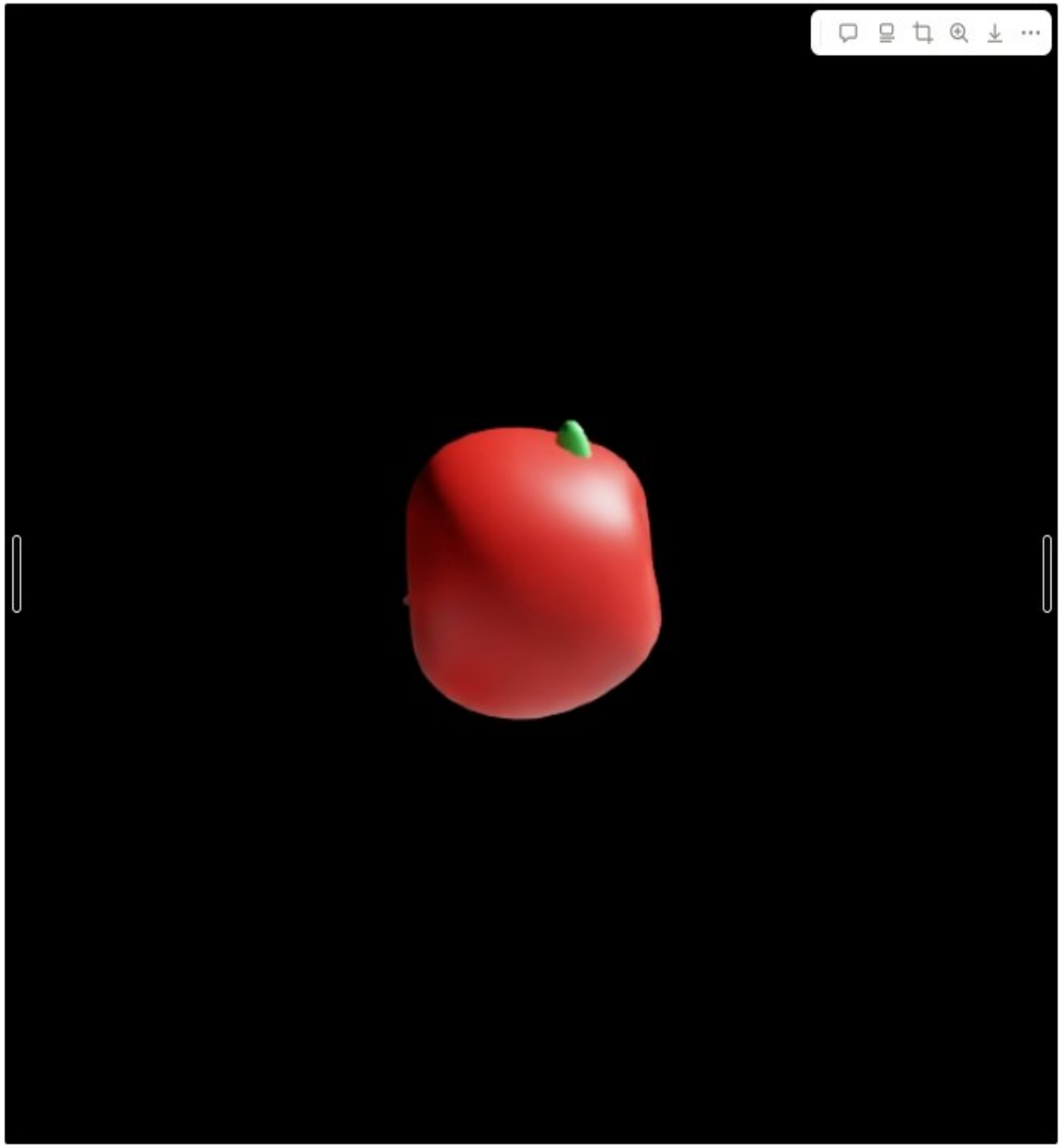
사과 에셋이 기본 구성으로 표면에 노란색이 그려져있음. 착색률 측정을 위해서 구분해야함



사과 에셋 전체를 빨간색으로 구성



패치가 튀어나오는 현상 발생



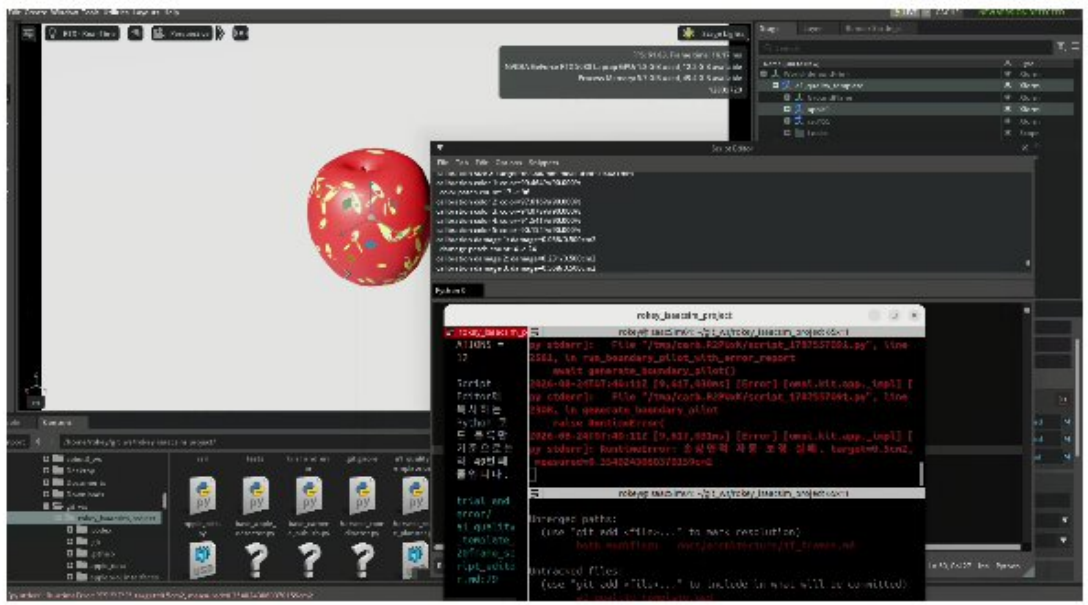
사과 표면에 **raycast**를 쏴서 표면 좌표를 측정

- 사과 표면에서 일정 수준 이상 벗어나는 **mask** 패치를 삭제
- 사과 표면과 완전히 겹치면 문제가 생길 수 있으므로 미세하게 위에 있게 설정

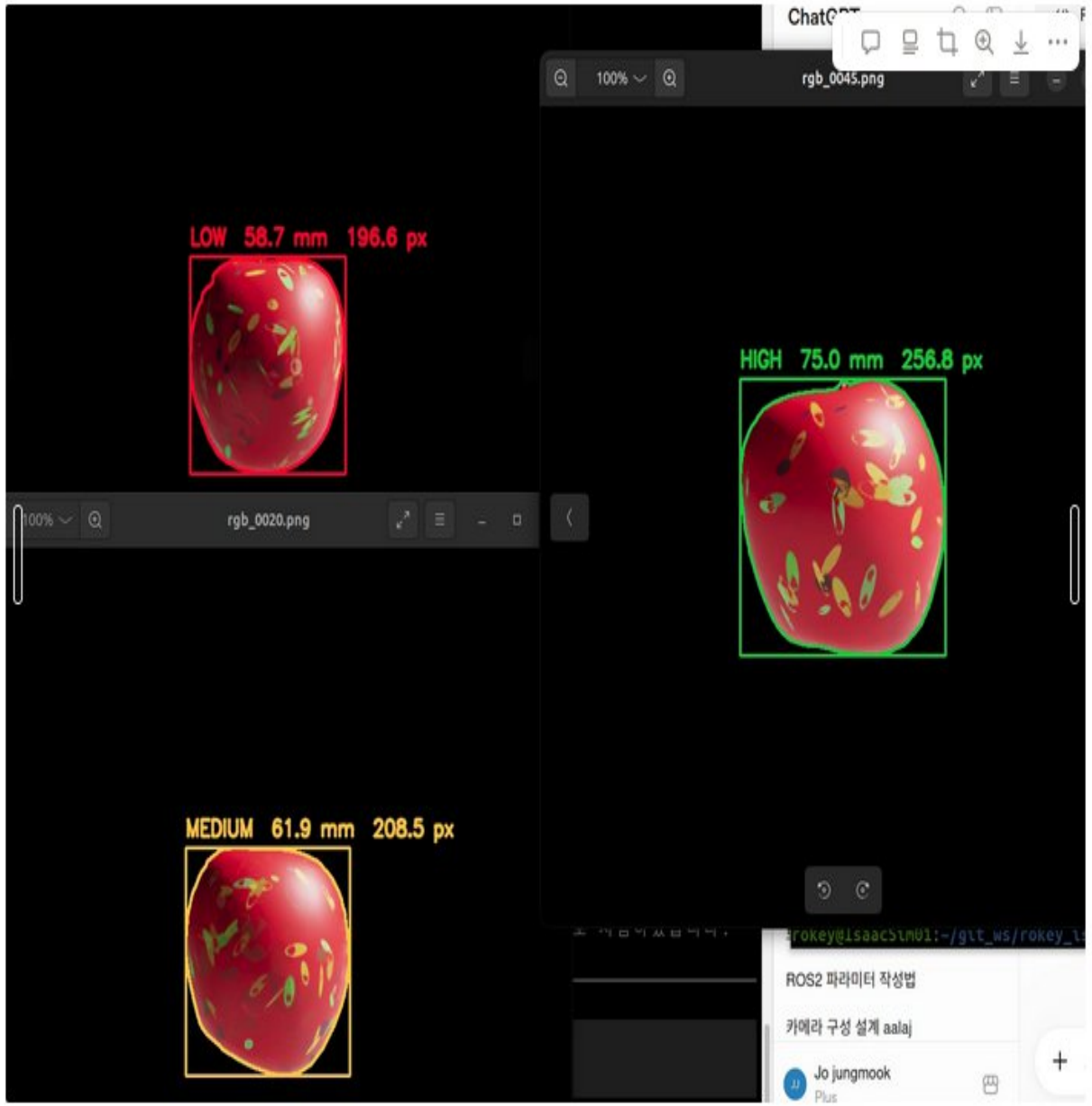
데이터를 검사해보니 900장은 정상적으로 있고 RGB·depth·CameraInfo·5종 mask가 모두 대응됩니다. 다만 생성 요청값과 실제 렌더 측정값은 크게 다릅니다. 착색률 실측은 전부 약 99.1~99.9%, 손상면적은 0~0.176cm², 직경은 약 54.1~74.3mm라서 현재 데이터만으로는 60/80%, 1.0/2.5cm², 75mm 경계를 충분히 학습·검증할 수 없습니다.

```
nhandled attribute type VtArray<std::string> (prim attribute: omni:rtx:material:db:fl
r:ior_constant)
경계값 파일럿 180장 생성 시작: /home/rokey/git_ws/rokey_isaacsim_project/dataset/appl
ity_boundary_pilot/run_20260824_155221
  calibration size 1: target=58.000mm measured=55.223mm
  calibration size 2: target=58.000mm measured=57.786mm
2026-08-24T06:52:23Z [6,748,899ms] [Warning] [rtx.scenedb.plugin] /World/a1_quality_t
e/apple1/QualityPatches/GeneratedBoundaryPatches/NonTargetPatch_05: normal's facevary
de changes between first frame and animated frames. This mostly is because first fram
nes face varying normal, uses point cache to animate, but doesn't include animated no
ata. Engine fallbacks to smooth and per-vertex normal.
  calibration color 1: color=99.601%/90.000%
  calibration color 2: color=99.014%/90.000%
  calibration color 3: color=99.179%/90.000%
  calibration color 4: color=98.903%/90.000%
  calibration color 5: color=96.979%/90.000%
  calibration color 6: color=96.880%/90.000%
```

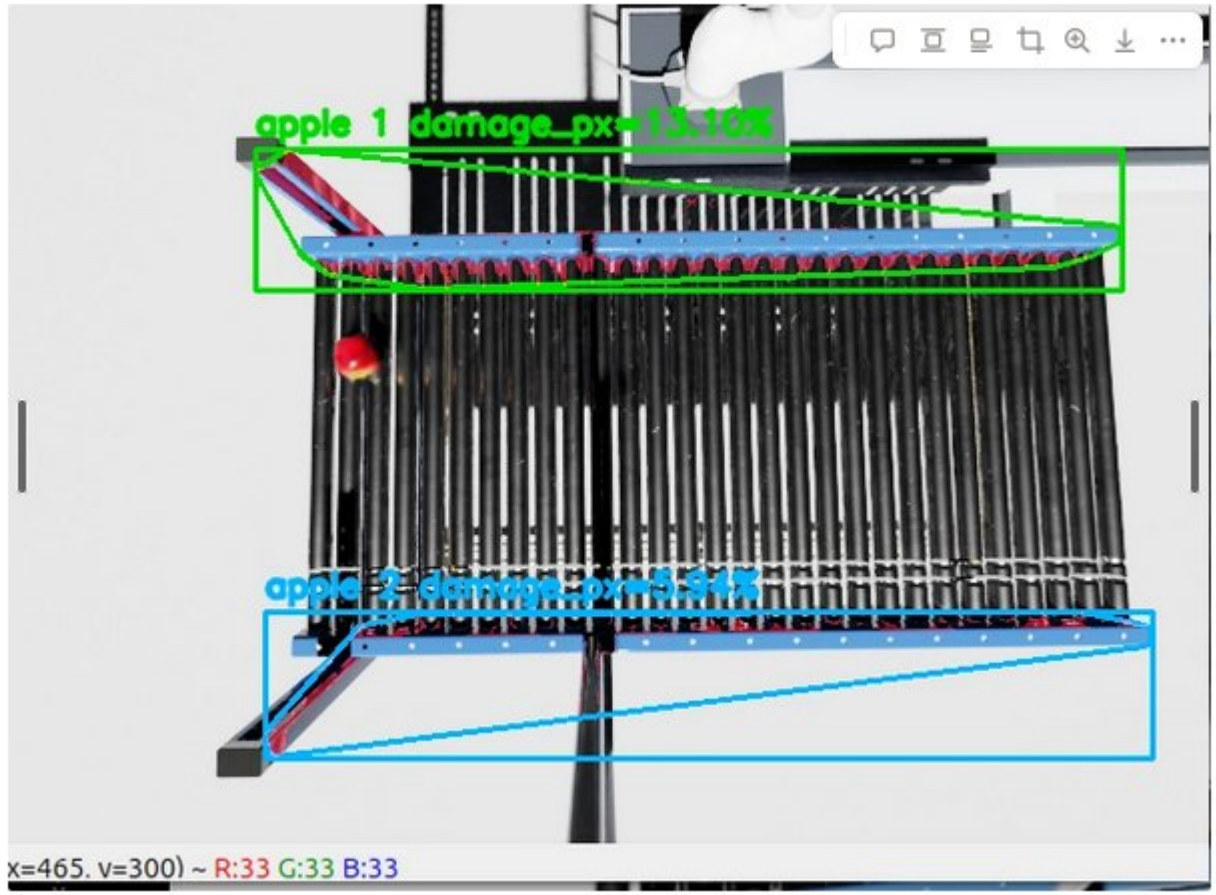
착색률이 경계값 근처로 고정되지 않아서 프로그램 정지하는 현상 ▼



Opencv 크기 분류 확인

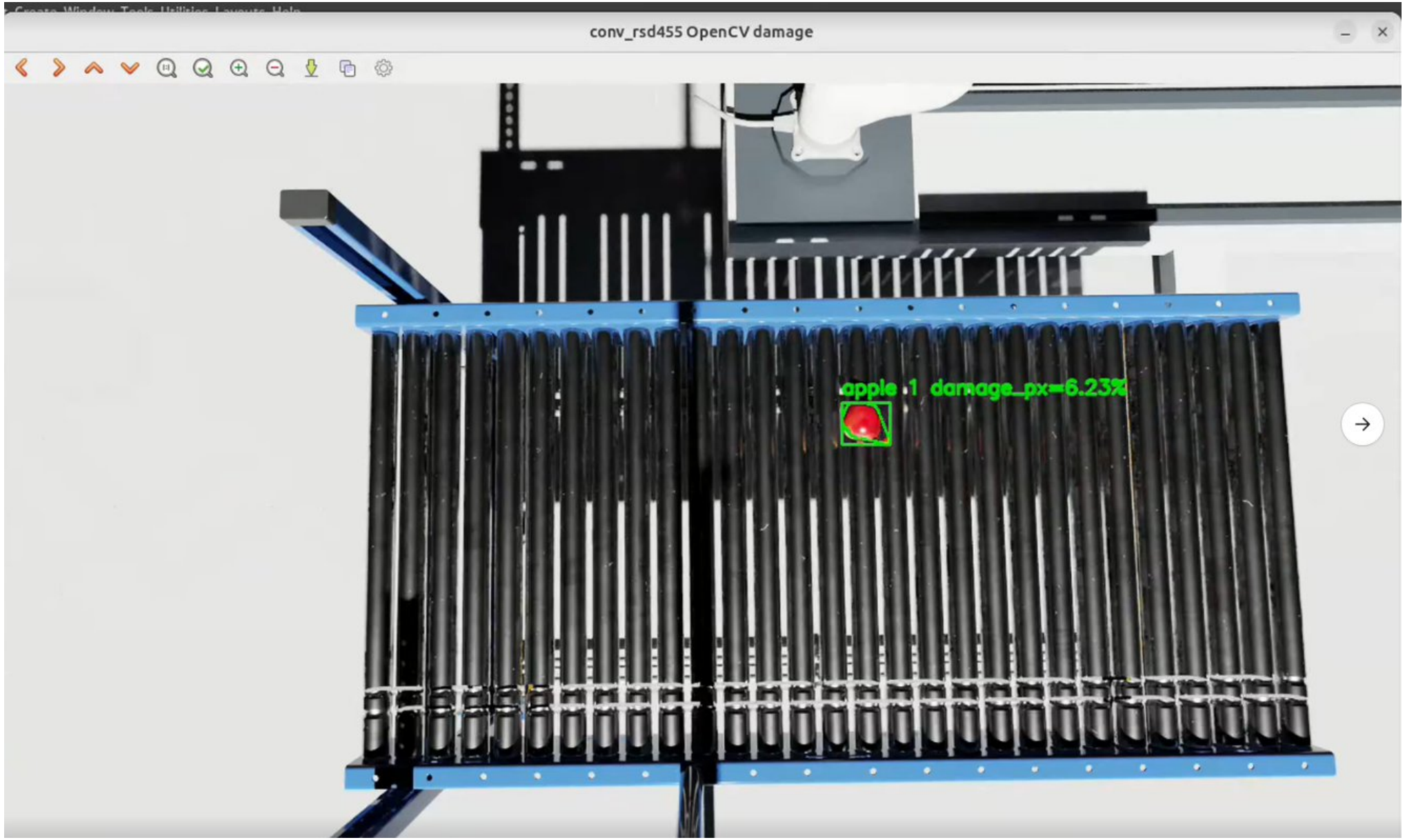


conv rsd455 opencv로 진행



컨베이어에 빨간 부분도 인식하는 현상

롤러 부분만 찍을 수 있도록 카메라 위치 조정



롤러에 비친 사과의 빨간색도 인식하는 문제 -> 롤러 재질을 변경
(비친 사과의 빨간색인 것을 부분을 데미지 면적으로 인식)

착색률 기준을 80으로 둔 이유 : 사과 밑쪽 그림자 지는 부분이 검게 검출돼서 100%의 빨간 사과로 인식 불가

지금 현재는 OpenCV 기반 착색률은 학습 모델이 아니라 수동으로 설정한 색상 조건으로 계산
사과 표면 영역(`apple_mask`)을 측정

HSV의 Hue·Saturation·Value 조건과 RGB 채널 비율로 '목표 적색' 픽셀을 분류
반사나 너무 어두운 영역은 `ignore_mask`로 제외

최종 착색률은 다음처럼 계산
착색률 = 착색 조건을 만족한 픽셀 수 / 유효한 사과 표면 픽셀 수그 결과에 80%/60% 등급 경계를 적용

따라서 현재 시스템은 "AI가 잘 익은 색을 학습해 판단"하는 구조가 아니라, "사람이 정한 적색 범위에 들어가는 픽셀 비율을 측정"하는 구조이다.
Hue: `0~10` 또는 `170~179`
Saturation: `40~255`
Value: `20~255`

다만, 합성 데이터를 다시 사용한다면…
다양한 착색률의 사과를 생성
RGB 이미지와 함께 정답 mask를 저장
사과 전체 mask
적색 착색 mask
반사·그림자 제외 mask
조명, 사과 재질·크기·자세, 카메라 노이즈 등을 다양화

자체 평가 의견

사전 기획 관점에서 결과물의 완성도 평가
10점

잘한 점: 프로젝트 초기부터 md파일을 구축하며 탄탄하게 개발함
아쉬운 점: 다음에는 사과 뿐 아니라 다양한 과일도 수확해보고 싶다.

추후 개선할 점
이번 프로젝트에서는 수직적인 확장을 먼저 진행했는데, 다음 프로젝트에서는 수평적인 확장을 먼저 진행해서 개발해보고 싶다.

학습 성과
ROS 2 통신에 대한 뜻깊은 이해
설계 과정부터 시작하는 개발과, 실제 적용하며 디버깅
문서화 / 기록하며 한 번의 프로젝트에서 그치지 않고 재현성 확보